

CAPA E CONTRACAPA



**AUTARQUIA DE ENSINO SUPERIOR DE ARCOVERDE
CENTRO DE ENSINO SUPERIOR DE ARCOVERDE
CURSO SUPERIOR EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

Trampou

Relatório Técnico

HIGOR FERNANDO FREITAS DE CARVALHO

HIGOR FERNANDO FREITAS DE CARVALHO

Trampou

Relatório Técnico apresentado como requisito parcial para a disciplina de Projeto de Desenvolvimento de Sistemas Web, do curso superior em Análise em Desenvolvimento de Sistemas, sob orientação do Profº Dennys Cavalcanti Carvalho.

HISTÓRICO DE REVISÃO

Data	Versão	Descrição
17/04/2026	1.0	Elaboração dos primeiros conteúdos para implementação no documento.
24/04/2026	1.1	Elaboração dos Requisitos Funcionais e do Diagrama de Caso de Uso
01/05/2026	1.2	Definição da Arquitetura de 3 Camadas e Mapeamento Objeto-Relacional (ORM)
08/05/2026	1.3	Criação dos Wireframes de baixa fidelidade e adição do diagrama de fluxo de navegação
15/05/2026	1.4	Desenvolvimento dos Mockups de alta fidelidade e refinamento da identidade visual
12/06/2026	1.5	Apresentação em sala.

SUMÁRIO

1 DOCUMENTO DE DEFINIÇÃO DE ESCOPO (DDE)	5
1.1. INTRODUÇÃO	5
1.2. VISÃO GERAL DO DOCUMENTO	5
1.3. IDENTIFICAÇÃO DO PROJETO	5
1.4. OBJETIVOS DO PROJETO	5
1.5. JUSTIFICATIVA	6
1.6. IDENTIFICAÇÃO DOS REQUISITOS	7
1.6.1. Prioridades dos Requisitos	7
1.7. ESCOPO DO PRODUTO E ENTREGÁVEIS	7
1.7.1. Funcionalidades Previstas	7
1.7.2. Entregáveis	8
1.8. PREMISSAS E RESTRIÇÕES	8
1.8.1. Premissas	8
1.8.2. Restrições	8
1.9. CRITÉRIOS DE ACEITAÇÃO DO PROJETO	8
1.10. EXCLUSÕES DO ESCOPO	9
1.11. STAKEHOLDERS ENVOLVIDOS	9
1.12. RISCOS INICIAIS	9
2 DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS (ERS)	10
2.1. REQUISITOS FUNCIONAIS	10
2.2. REQUISITOS NÃO FUNCIONAIS	12
2.3. REGRAS DE NEGÓCIO	13
3 DOCUMENTO DE ESPECIFICAÇÃO DE MODELAGEM (DEM)	14
3.1 MODELAGEM DE DADOS	14
3.1.1 Entidade-Relacionamento	15
3.1.2 Dicionário de Dados	19
3.2 MODELAGEM COMPORTAMENTAL	19
3.2.1 Diagrama de Sequência	20
3.2.2 Diagrama de Estados	21
3.3. MODELAGEM ESTRUTURAL	22
3.3.1 Diagrama de Caso de Uso	22
3.3.2 Diagrama de Classes	23
3.3.3 Diagrama de Componentes	24
3.3.4 Diagrama de Arquitetura	26
3.4 MAPEAMENTO OBJETO-RELACIONAL (ORM)	27
4 DOCUMENTO DE ESPECIFICAÇÃO DE INTERFACES (DEI)	29
4.1. WIREFRAMES	29
4.2. MOCKUPS	36
4.3. FLUXO DE NAVEGAÇÃO	44
5 DOCUMENTAÇÃO TÉCNICA	48
5.1. ARQUITETURA DO SISTEMA	48
5.1.1. Segmentação da Arquitetura	48

5.2. TECNOLOGIAS UTILIZADAS.....	49
5.2.1 Frontend	49
5.2.2. Backend	49
5.2.3. Banco de Dados	49
5.2.4. Ferramentas de Apoio	49
5.2.5. Padrões Adotados	50
5.2.6. Boas Práticas e Convenções	50
5.2.7. Requisitos de Infraestrutura	51
5.2.8. APIs e Integrações	51
5.2.9. Caracterização da API	51
5.3. REPOSITÓRIO E CÓDIGO-FONTE.....	51
6. MANUAL DO USUÁRIO	52
7. REFERÊNCIAS	65
8. APÊNDICE	66

DDE

1 DOCUMENTO DE DEFINIÇÃO DE ESCOPO (DDE)

1.1 INTRODUÇÃO

O projeto Trampou surge como uma plataforma web inovadora desenvolvida para modernizar e centralizar a conexão entre empregadores e trabalhadores. Arquetado como uma solução digital escalável, o sistema visa atender às necessidades de polos comerciais em municípios de pequeno e médio porte que ainda carecem de infraestrutura tecnológica voltada para o recrutamento e seleção.

A plataforma substitui o fluxo tradicional e analógico de entrega de currículos em papel por um ambiente virtual dinâmico, onde empresas locais podem gerenciar suas oportunidades de forma estruturada, e cidadãos podem criar perfis digitais para se candidatar às vagas com agilidade e segurança.

1.2 VISÃO GERAL DO DOCUMENTO

Este documento apresenta o escopo detalhado do Trampou, uma plataforma web destinada à gestão e divulgação de vagas de emprego locais. Ele serve como um mapa do projeto, contendo as definições de escopo, os objetivos primários e secundários, as justificativas para sua construção, bem como os entregáveis e restrições que guiarão todo o ciclo de desenvolvimento do software.

1.3 IDENTIFICAÇÃO DO PROJETO

Nome do Projeto: Trampou

Autor: Higor Fernando Freitas de Carvalho

1.4 OBJETIVOS DO PROJETO

Objetivo Geral: Desenvolver uma plataforma web responsiva para centralizar e gerenciar a oferta de vagas de emprego no município de Buíque, conectando empresas locais a candidatos de forma ágil e estruturada.

Objetivos Específicos:

- Levantar os requisitos funcionais e regras de negócio junto a representantes do comércio local.
- Projetar a modelagem do banco de dados para gerenciar usuários, empresas e vagas.
- Desenvolver uma interface *mobile-first* para facilitar o acesso da população através de smartphones.
- Implementar um módulo de cadastro simplificado de currículos e um painel de triagem para os empregadores.
- Validar a aplicação por meio de testes de usabilidade com um grupo de lojistas e candidatos voluntários.

1.5 JUSTIFICATIVA

Conforme levantamento observacional e análise do comportamento do comércio local em municípios de pequeno e médio porte, constatou-se que atualmente a divulgação de vagas de emprego ocorre de maneira fragmentada e altamente informal. Dependendo frequentemente de avisos impressos colados em fachadas de lojas ou de publicações efêmeras em aplicativos de mensagens, esse cenário gera um problema duplo: o empresário perde tempo recebendo e organizando currículos físicos despadronizados, e o trabalhador muitas vezes não toma conhecimento das oportunidades disponíveis por não estar no local físico ou no grupo digital correto no momento exato do anúncio.

A centralização dessas informações por meio da plataforma Trampou reduz o tempo de ociosidade das vagas e otimiza o processo de recrutamento, especialmente para pequenos negócios que não possuem um setor formal de Recursos Humanos. Corroborando essa perspectiva, Chiavenato (2020) afirma que o uso da internet e de plataformas digitais no recrutamento (*e-recruiting*) confere maior agilidade, minimiza custos operacionais e democratiza o acesso às oportunidades, facilitando a triagem inicial sem a necessidade de uma estrutura complexa de RH. Sob o prisma tecnológico, Turban et al. (2021) enfatizam que sistemas de informação bem estruturados transformam a dinâmica das organizações e do mercado, permitindo que a tecnologia atue como um elo eficiente na conexão entre oferta e procura, eliminando barreiras de comunicação e garantindo a integridade dos dados compartilhados.

Sob a ótica da tecnologia, Laudon e Laudon (2021) destacam que sistemas de informação bem estruturados eliminam a assimetria de informações, conectando de forma direta, transparente e eficiente a oferta e a demanda em um mercado específico. Assim, o Trampou atua como uma ponte tecnológica essencial para o desenvolvimento econômico local.

1.6 IDENTIFICAÇÃO DOS REQUISITOS

Por convenção, os requisitos são referenciados pelo nome da subseção onde estão descritos, seguido do seu identificador, conforme o esquema abaixo:

- O requisito funcional [Cadastro de Usuários.RF-01] está localizado na subseção “Requisitos Funcionais”, dentro do bloco identificado como [RF-01].
- O requisito não funcional [Disponibilidade.NF-04] encontra-se na seção “Requisitos Não Funcionais de Confiabilidade”, no bloco identificado como [NF-04].

1.6.1 Prioridades dos Requisitos

Os requisitos do sistema são classificados em três níveis de prioridade:

- Essencial: indispensável para o funcionamento do sistema. Sem ele, o sistema não opera. Deve ser obrigatoriamente implementado.
- Importante: afeta a qualidade do funcionamento. O sistema pode ser utilizado sem esse requisito, mas de forma insatisfatória. Sua implementação é recomendada.
- Desejável: não interfere nas funcionalidades básicas. O sistema funciona bem sem ele. Pode ser incluído em versões futuras, caso não haja tempo para implementá-lo na versão atual.

1.7 ESCOPO DO PRODUTO E ENTREGÁVEIS

1.7.1 Funcionalidades previstas

- Mural digital público para listagem e busca de vagas por categoria.
- Criação de perfil digital do candidato.

- Painel administrativo restrito para a empresa gerenciar suas vagas e interessados.

1.7.2 Entregáveis

- Software completamente funcional hospedado em ambiente de produção.
- Relatório técnico completo incluindo requisitos, diagramas, modelos, protótipos, documentação técnica e manual de usuário.
- Código fonte disponibilizado pelo Github.

1.8 PREMISSAS E RESTRIÇÕES

1.8.1 Premissas

- Os usuários finais (tanto candidatos quanto empresas) possuirão acesso à internet e dispositivos móveis ou computadores.
- Haverá adesão inicial de comerciantes.
- Haverá busca por vagas de emprego pela plataforma.

1.8.2 Restrições

- O sistema será inicialmente desenvolvido apenas como aplicação Web (sem aplicativos nativos para Android/iOS nesta versão).
- O projeto deve ser concluído e apresentado dentro do cronograma do semestre letivo da instituição de ensino.
- O software será desenvolvido utilizando Next.js (React) no Frontend, garantindo alta performance e navegação fluida, juntamente com a biblioteca Tailwind CSS para a estilização responsiva. O backend será estruturado por meio de uma API RESTful desenvolvida em Python utilizando o framework FastAPI, responsável por processar as regras de negócio e realizar a comunicação com um banco de dados relacional PostgreSQL, garantindo a integridade e persistência estruturada das informações. A infraestrutura de apresentação estará hospedada na nuvem e o versionamento de código será feito via GitHub.

1.9 CRITÉRIOS DE ACEITAÇÃO DO PROJETO

- O sistema deve permitir que uma empresa publique uma nova vaga em menos de 3 minutos.
- O layout deve se comportar corretamente em telas de smartphones, tablets e desktops sem quebra de elementos estruturais.
- O banco de dados deve persistir e relacionar corretamente as entidades.
- Teste unitários e de integração deverão cobrir acima de 90% da solução.

1.10 EXCLUSÕES DO ESCOPO

- Não haverá sistema de chat ou mensagens diretas em tempo real entre empresa e candidato dentro da plataforma.
- O sistema não fará testes psicológicos, de perfil ou aplicação de provas de nivelamento técnico.
- Não haverá integração com gateways de pagamento (o uso do sistema será considerado gratuito nesta etapa de projeto acadêmico).

1.11 STAKEHOLDERS ENVOLVIDOS

Desenvolvedor: Higor Fernando Freitas de Carvalho.

Participantes: Moradores de Buíque e região em busca de emprego, Comerciantes e empresários locais.

Professor orientador: Prof. Dennys Cavalcanti Carvalho.

Testers: Carlos Eduardo da Silva Tenório.

1.12 RISCOS INICIAIS

- Baixo engajamento inicial para cadastrar as vagas durante a fase de testes reais.
- Períodos de sazonalidade da oferta de vagas.
- Lojistas podem postar anúncios incompletos ou com erros gramaticais, fugindo do padrão estético ideal.
- Grupos de Facebook ou WhatsApp continuarem sendo usados paralelamente ao Projeto.
- Usuários com conexões de internet instáveis ou aparelhos antigos podem enfrentar lentidão.

ERS

2 DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS (ERS)

2.1 REQUISITOS FUNCIONAIS

- [RF-01] Cadastro e Autenticação de Usuários:
O sistema deve permitir o registro e login de usuários, diferenciando os perfis em "Empresa" (via CNPJ) e "Candidato" (via CPF), além de disponibilizar a funcionalidade de recuperação de senha.

Atores: Empresa e Candidato.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O usuário preenche os campos obrigatórios (nome, e-mail, senha, documento), o sistema valida a formatação do CPF/CNPJ, e o usuário recebe a confirmação de criação de conta.

- [RF-02] Gestão de Perfil da Empresa:
O sistema deve permitir que a empresa complete e gerencie seu perfil público, adicionando nome fantasia, logotipo, descrição do negócio e endereço comercial.

Atores: Empresa.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

- Critério de aceitação: A empresa preenche o formulário de perfil, clica em salvar, e o sistema exibe os dados atualizados em sua página pública para os candidatos.

- [RF-03] Criação de Perfil Digital do Candidato:
O sistema deve permitir que o candidato preencha seu currículo digital de forma estruturada, incluindo dados pessoais, meios de contato, experiências profissionais, escolaridade e habilidades.

Atores: Candidato

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O candidato insere seus dados nos campos estruturados, salva o formulário, e o sistema gera o seu perfil digital padronizado, bloqueando a submissão se campos obrigatórios de contato estiverem vazios

- [RF-04] Gestão de Anúncios de Vagas:
O perfil Empresa deve conseguir criar, editar, pausar e encerrar anúncios de vagas, definindo título, descrição detalhada, requisitos obrigatórios, salário (opcional) e carga horária.

Atores: Empresa.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O empregador preenche os dados da nova oportunidade, confirma a ação, e a vaga é listada imediatamente no mural público com o status "Ativa"

- [RF-05] Busca e Filtro de Vagas:
O sistema deve disponibilizar um mural público onde visitantes e candidatos possam buscar vagas ativas por palavras-chave e filtrar por categorias de atuação (ex: Vendas, Serviços Gerais, Administrativo).

Atores: Candidato e Visitante

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O usuário digita um termo na barra de busca ou seleciona um filtro, e a tela é atualizada em menos de 3 segundos apenas com as vagas correspondentes aos critérios escolhidos.

- [RF-06] Processo de Candidatura:
O perfil Candidato deve poder se candidatar a uma vaga ativa com apenas um clique, o que vinculará automaticamente o seu Perfil Digital à vaga selecionada pela empresa.

Atores: Candidato.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O candidato clicar no botão "Candidatar-se", o sistema verifica se o perfil está completo, registra a inscrição na base de dados e exibe a mensagem "Candidatura realizada com sucesso".

- [RF-07] Triagem e Gestão de Candidatos (Funil):
A Empresa deve poder visualizar a lista de inscritos em suas vagas, acessar o Perfil Digital detalhado de cada candidato e alterar o status de triagem no processo seletivo (ex: "Novo", "Em análise", "Entrevista", "Aprovado", "Recusado").

Atores: Empresa.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: A empresa acessa o painel da vaga, visualiza a lista de nomes, seleciona um candidato e altera seu status em um menu suspenso, salvando o progresso da seleção.

- [RF-08] Notificações de Status:
O sistema deve notificar o candidato, através do seu painel interno, sempre que a empresa atualizar o seu status na etapa de triagem de uma vaga.

Atores: Candidato.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

- Critério de aceitação: Quando a empresa muda o status do candidato (ex: de "Em análise" para "Entrevista"), um ícone de alerta visual aparece no painel do candidato informando a atualização.

2.2 REQUISITOS NÃO FUNCIONAIS

- [NF-01] (Usabilidade):
A interface do sistema deve ser construída de forma responsiva, garantindo total funcionalidade, legibilidade e adaptação visual em telas de smartphones, tablets e desktops (padrão *mobile-first*).

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de medição: Teste de layout em diferentes resoluções utilizando o Chrome DevTools, confirmando a ausência de quebras de elementos visuais.

- [NF-02] (Desempenho):
O tempo de resposta para o carregamento do mural público de vagas e dos resultados de busca não deve exceder 3 segundos em conexões de internet 4G padrão.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

- Critério de medição: Teste de performance com ferramentas de medição (Lighthouse, GTmetrix) apontando carregamento da página inferior a 3 segundos.

- [NF-03] (Segurança dos Dados):
O sistema deve armazenar as senhas dos usuários utilizando criptografia forte (hash) e garantir rotinas de proteção contra acessos não autorizados nas rotas e painéis restritos.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de medição: Verificação direta no banco de dados confirmando que as senhas não estão em texto plano, e teste de bloqueio (erro 401/403) ao tentar acessar uma URL restrita sem estar logado.

- [NF-04] (Compatibilidade):
O software web deve ser plenamente compatível e funcional nas versões recentes dos principais navegadores do mercado (Google Chrome, Mozilla Firefox, Microsoft Edge e Safari).

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

- Critério de medição: Execução dos fluxos principais (cadastro e candidatura) em cada um dos navegadores listados sem ocorrência de erros no console.

2.3 REGRAS DE NEGÓCIO

- [RN-01] Limitação de Publicação:
Apenas usuários autenticados e com o perfil "Empresa" validado possuem permissão para criar, editar e gerenciar anúncios de vagas no mural.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- [RN-02] Restrição de Candidatura por Perfil Incompleto:
Um usuário com perfil "Candidato" só poderá efetuar sua candidatura em uma vaga se o seu Perfil Digital possuir todos os campos obrigatórios (como contato e experiências) previamente preenchidos.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- [RN-03] Candidatura Única:
O sistema deve bloquear candidaturas duplicadas, garantindo que um candidato só possa se inscrever uma única vez para a mesma vaga enquanto ela estiver ativa.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- [RN-04] Validação de Documentos de Identificação:
No momento do cadastro, o sistema deve validar matematicamente os números informados de CPF (para candidatos) e CNPJ (para empresas), bloqueando a criação de contas com documentos inválidos.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- [RN-05] Expiração Automática de Vagas Antigas:
Para manter o mural atualizado, vagas publicadas que não receberem edições ou não forem encerradas manualmente pela empresa serão automaticamente pausadas pelo sistema após 30 dias de sua publicação original.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

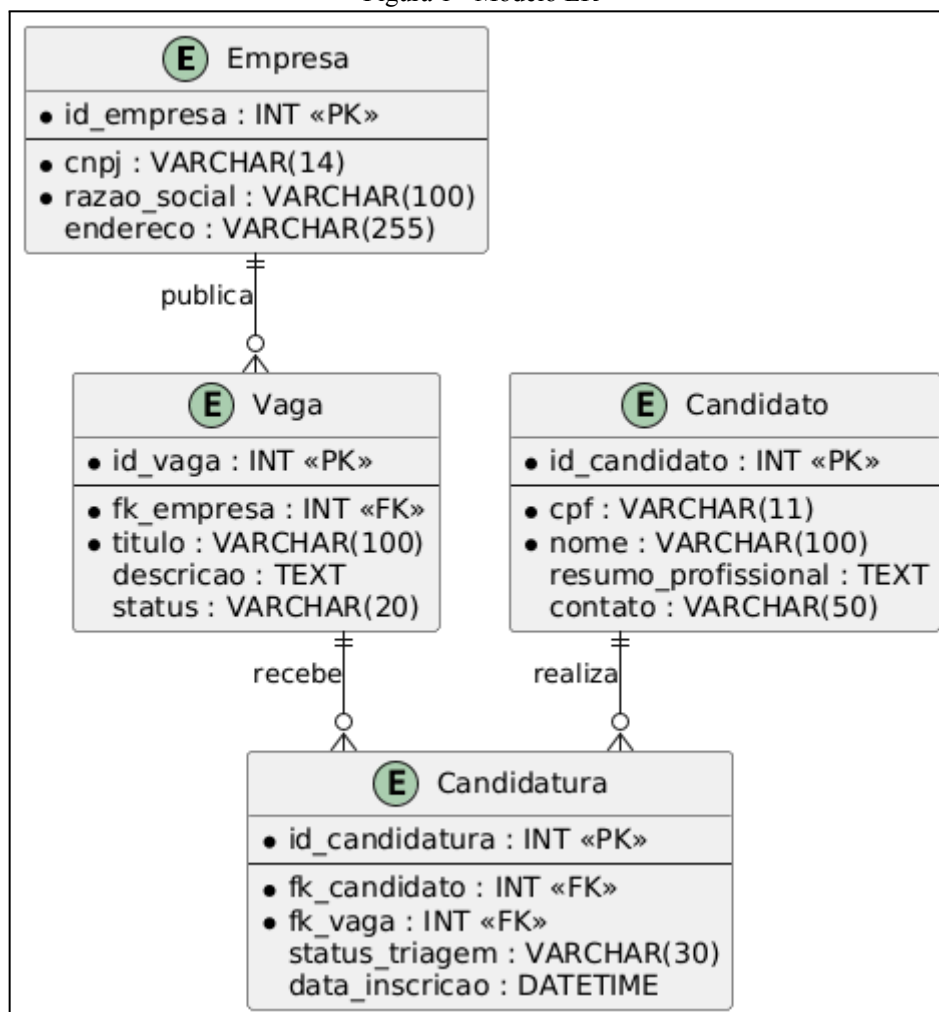
DEM

3 DOCUMENTO DE ESPECIFICAÇÃO DE MODELAGEM (DEM)

3.1 MODELAGEM DE DADOS

3.1.1 Entidade-Relacionamento

Figura 1 - Modelo ER



Fonte: Produzido pelo autor

3.1.2 Dicionário de Dados

Tabela Vaga

Nome da Coluna	Tipo de Dados	Tamanho	Restrições	Valor Padrão	Descrição
id_vaga	INT	-	PK, Auto Increment	-	Identificador único da vaga
fk_empresa	INT	-	FK, Not Null	-	Chave estrangeira ligando à tabela Empresas
titulo	VARCHAR	100	Not Null	-	Título ou cargo da oportunidade de emprego
descricao	TEXT	-	Not Null	-	Descrição detalhada das atividades e requisitos
salario	DECIMAL	10,2	-	NULL	Valor da remuneração ofertada (opcional)
status	VARCHAR	20	Not Null	'Ativa'	Estado atual da vaga (Ativa, Pausada, Encerrada)
data_criacao	DATETIME	-	Not Null	CURRENT_TIMESTAMP	Data e hora em que a vaga foi publicada no mural

Fonte: Produzido pelo autor.

Tabela 2: Candidatos

Nome da Coluna	Tipo de Dados	Tamanho	Restrições	Valor Padrão	Descrição
id_candidato	BIGINT	-	PK, NOT NULL	AUTO_INCREMENT	Identificador único do candidato.
nome	VARCHAR	100	NOT NULL	-	Nome completo do usuário.
cpf	VARCHAR	11	UNIQUE, NOT NULL	-	Cadastro de Pessoa Física.
email	VARCHAR	100	UNIQUE, NOT NULL	-	Endereço de e-mail para login e contato.
senha	VARCHAR	255	NOT NULL	-	Senha criptografada (hash) para acesso.
resumo_profissional	TEXT	-	NULL	-	Breve descrição da carreira e habilidades do candidato.
created_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	Data e hora de criação do registro no sistema.

Fonte: Produzido pelo autor.

Tabela 3: Empresas

Nome da Coluna	Tipo de Dados	Tamanho	Restrições	Valor Padrão	Descrição
id_empresa	BIGINT	-	PK, NOT NULL	AUTO_INCREMENT	Identificador único da empresa.
razao_social	VARCHAR	150	NOT NULL	-	Nome oficial ou razão social da organização.
cnpj	VARCHAR	14	UNIQUE, NOT NULL	-	Cadastro Nacional da Pessoa Jurídica.
email	VARCHAR	100	UNIQUE, NOT NULL	-	E-mail corporativo para gestão da conta.
senha	VARCHAR	255	NOT NULL	-	Senha criptografada (hash) para acesso ao painel.
descricao	TEXT	-	NULL	-	Perfil descritivo e informações sobre o negócio.
created_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	Data e hora de registro da empresa no portal.

Fonte: Produzido pelo autor.

Tabela 4: Candidaturas

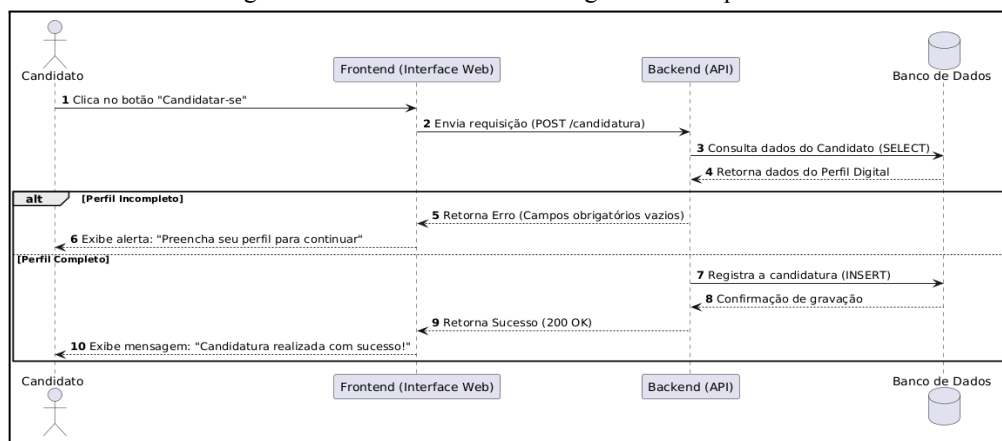
Nome da Coluna	Tipo de Dados	Tamanho	Restrições	Valor Padrão	Descrição
id_candidato	BIGINT	-	PK, FK, NOT NULL	-	Chave estrangeira referenciando tb_candidatos.
id_vaga	BIGINT	-	PK, FK, NOT NULL	-	Chave estrangeira referenciando tb_vagas.
status_triagem	VARCHAR	30	NOT NULL	'EM_ANALISE'	Etapa atual do candidato no processo seletivo.
data_inscricao	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	Data e hora exatas da submissão da candidatura.

Fonte: Produzido pelo autor.

3.2 MODELAGEM COMPORTAMENTAL

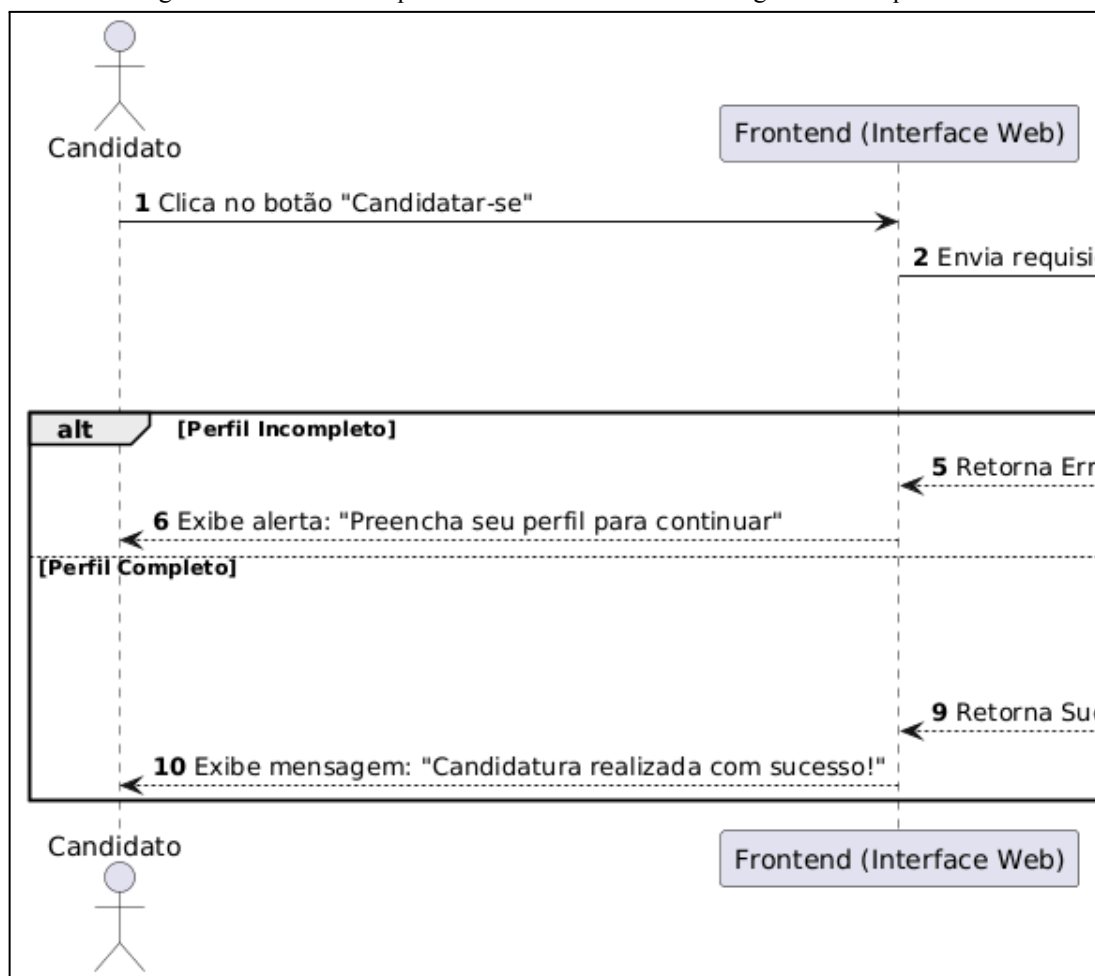
3.2.1 Diagrama de Sequência

Figura 2 - Panorama Geral do Diagrama de Sequência



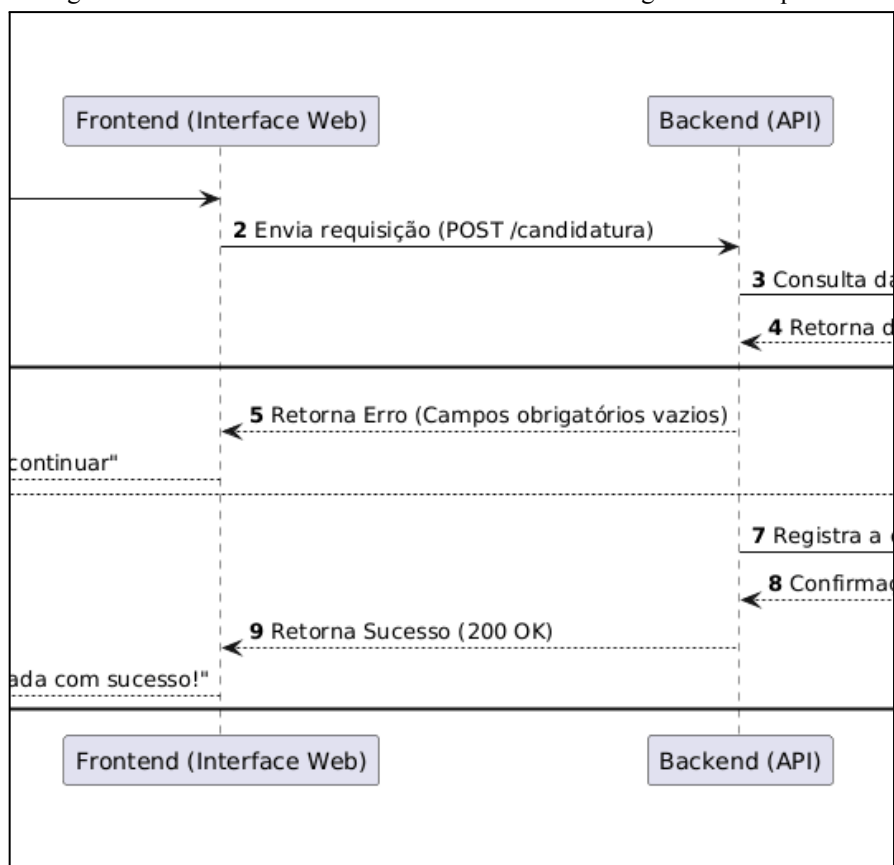
Fonte: Produzido pelo autor

Figura 3 - Dimensão Esquerda do Panorama Geral do Diagrama de Sequência



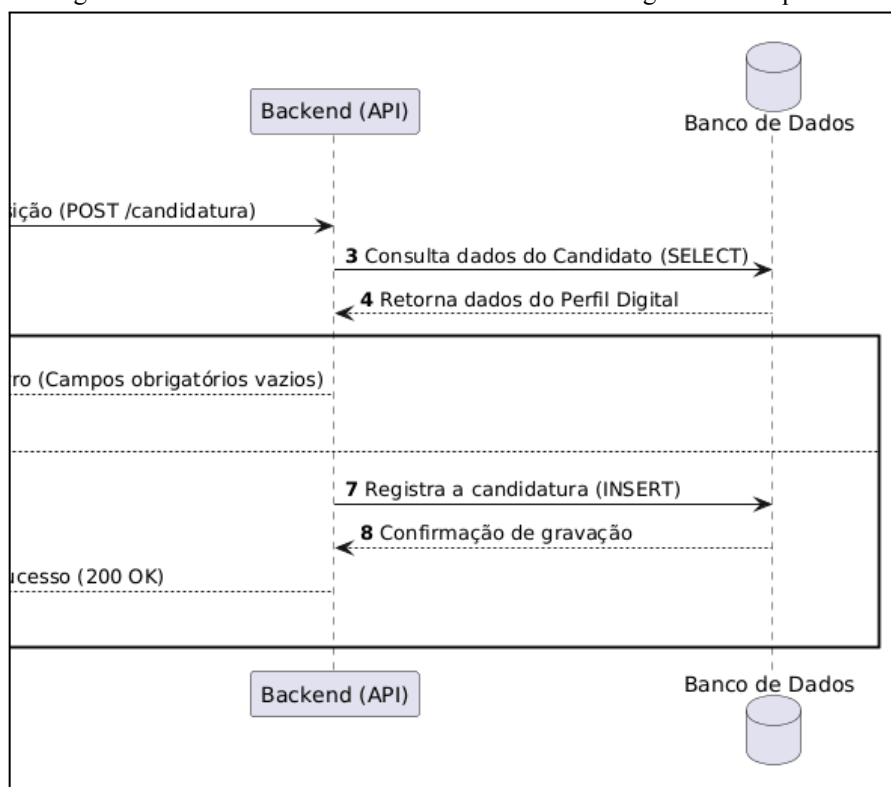
Fonte: Produzido pelo autor

Figura 4 - Dimensão Central do Panorama Geral do Diagrama de Sequência



Fonte: Produzido pelo autor

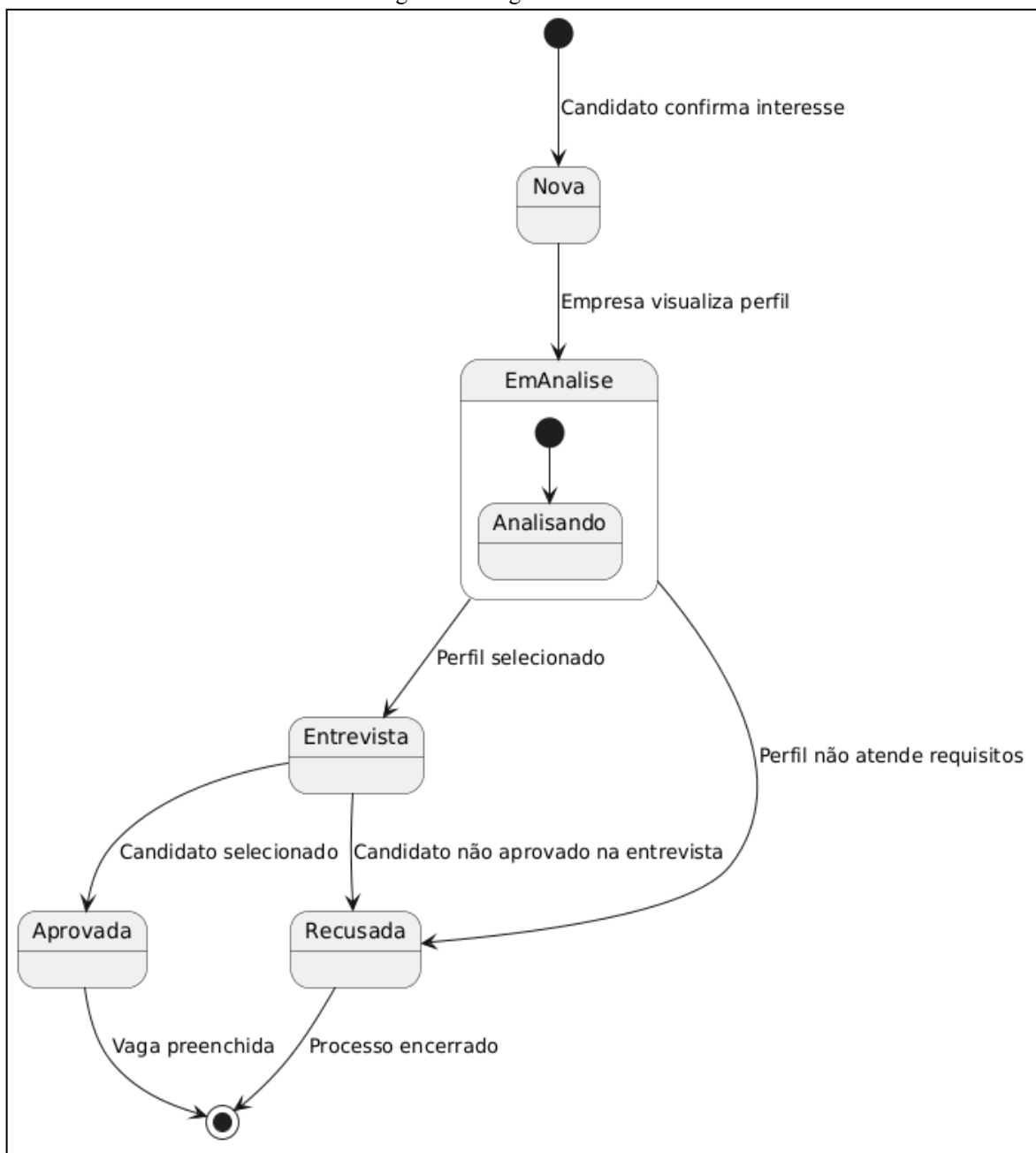
Figura 5 - Dimensão Direita do Panorama Geral do Diagrama de Sequência



Fonte: Produzido pelo autor

3.2.2 Diagrama de Estados

Figura 6 - Diagrama de Estados

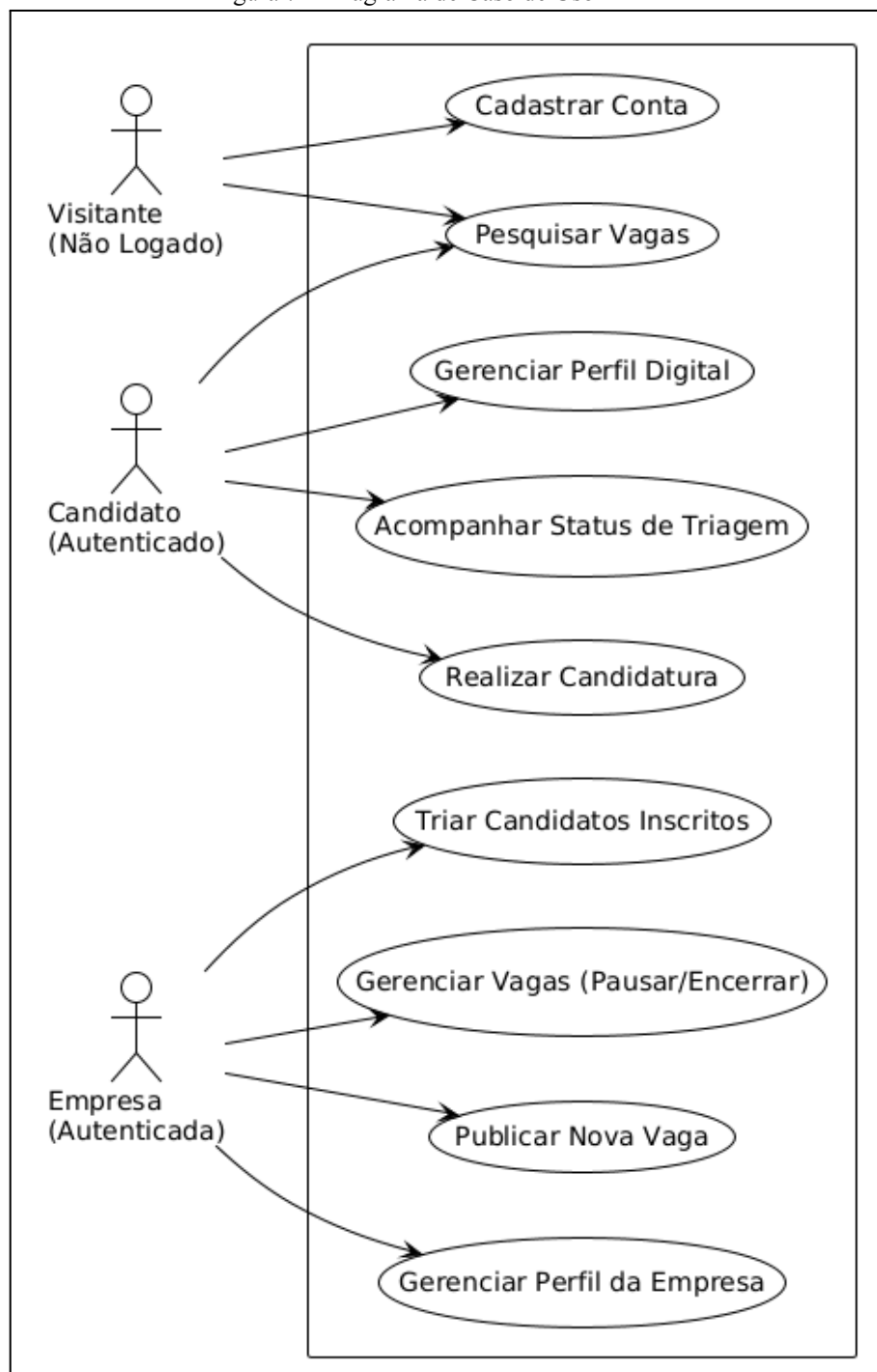


Fonte: Produzido pelo autor

3.3 MODELAGEM ESTRUTURAL

3.3.1 Diagrama de Caso de Uso

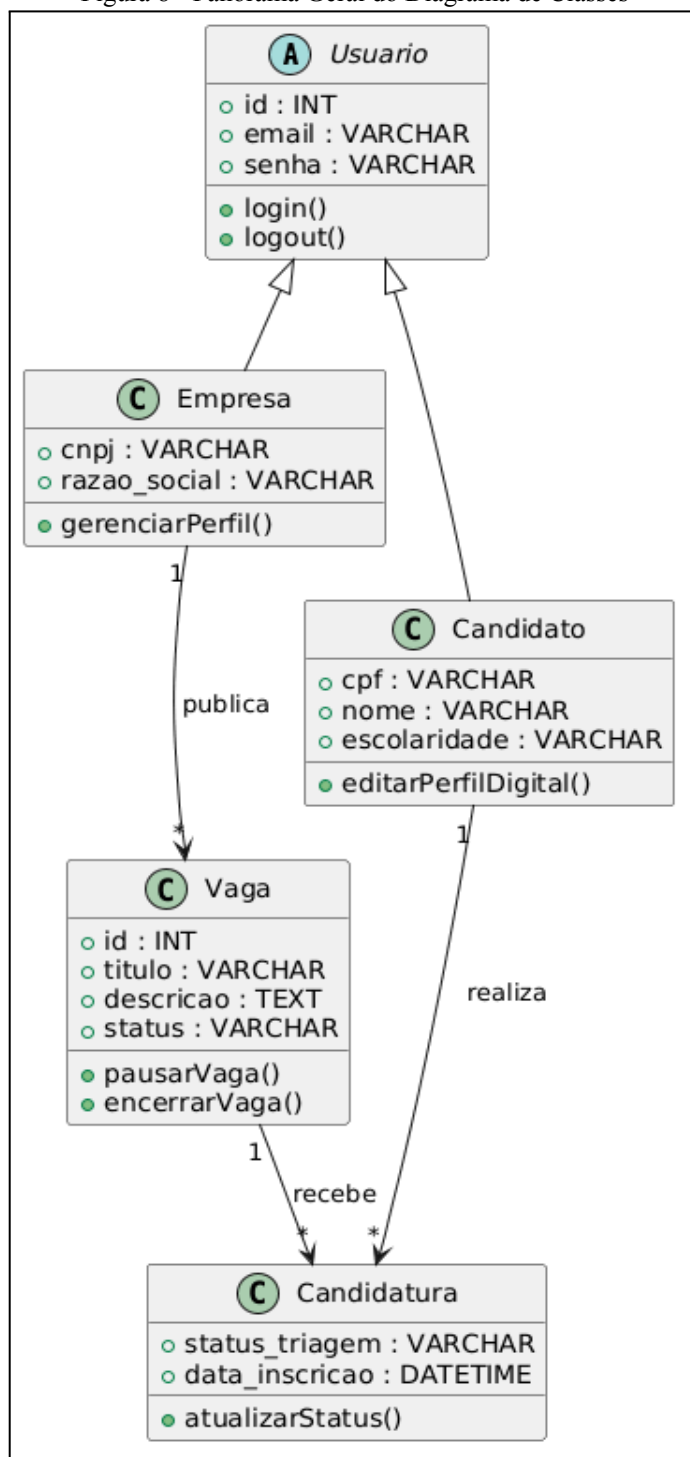
Figura 7 - Diagrama de Caso de Uso



Fonte: Produzido pelo autor

3.3.2 Diagrama de Classes

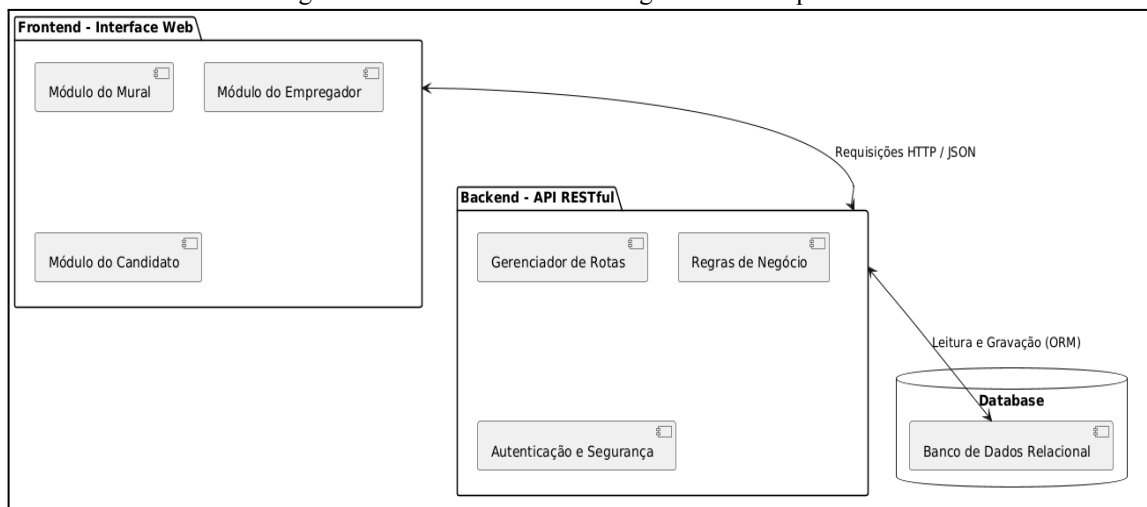
Figura 8 - Panorama Geral do Diagrama de Classes



Fonte: Produzido pelo autor

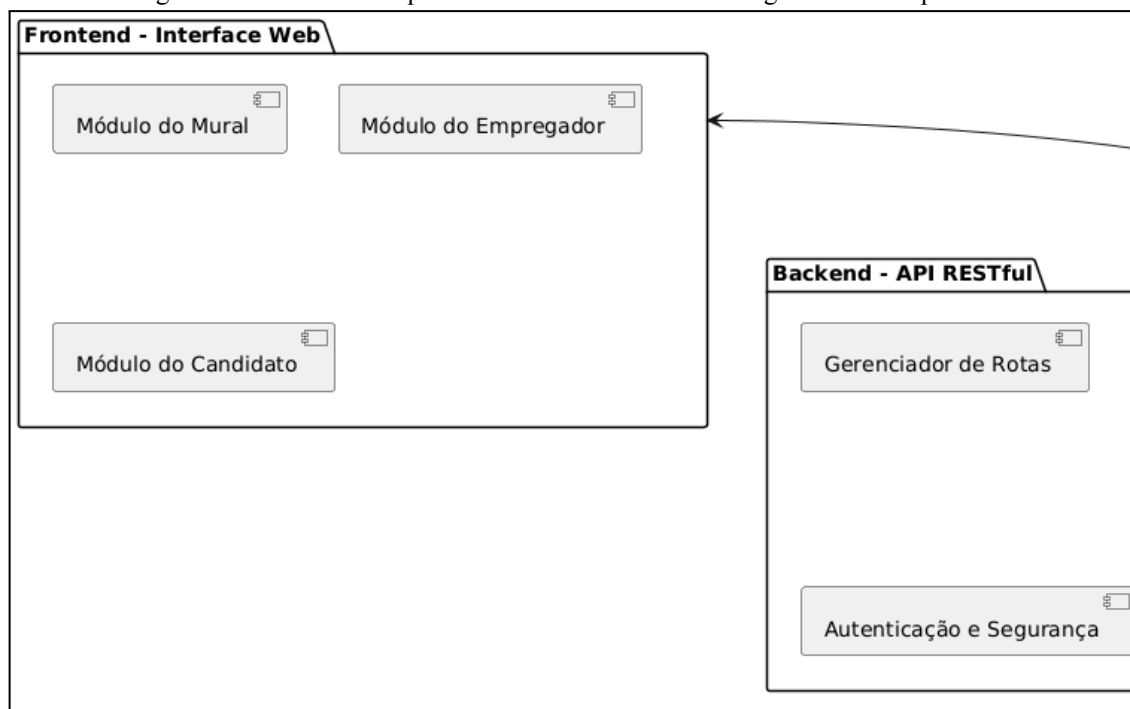
3.3.3 Diagrama de Componentes

Figura 9 - Panorama Geral do Diagrama de Componentes



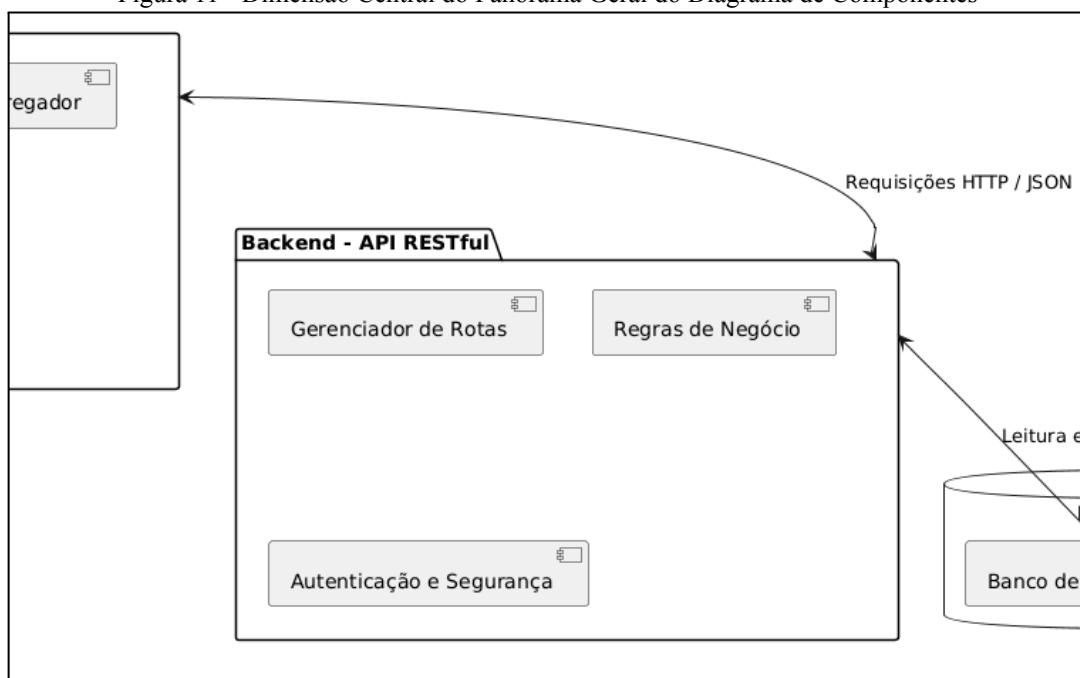
Fonte: Produzido pelo autor

Figura 10 - Dimensão Esquerda do Panorama Geral do Diagrama de Componentes



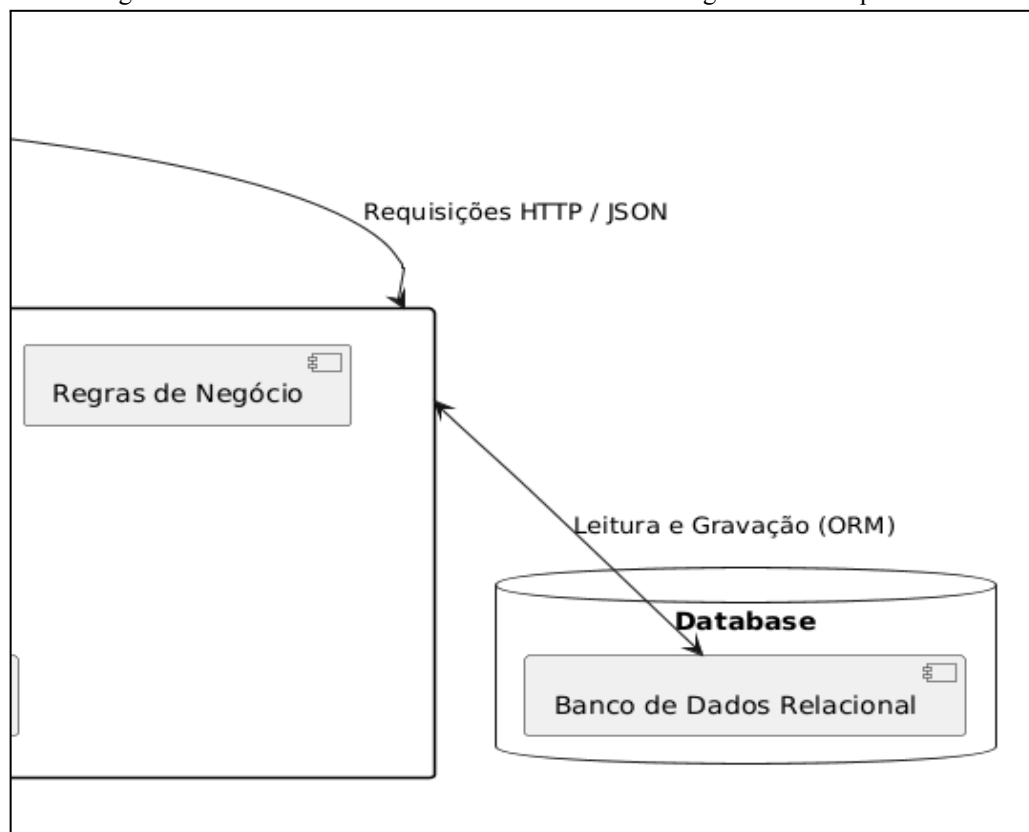
Fonte: Produzido pelo autor

Figura 11 - Dimensão Central do Panorama Geral do Diagrama de Componentes



Fonte: Produzido pelo autor

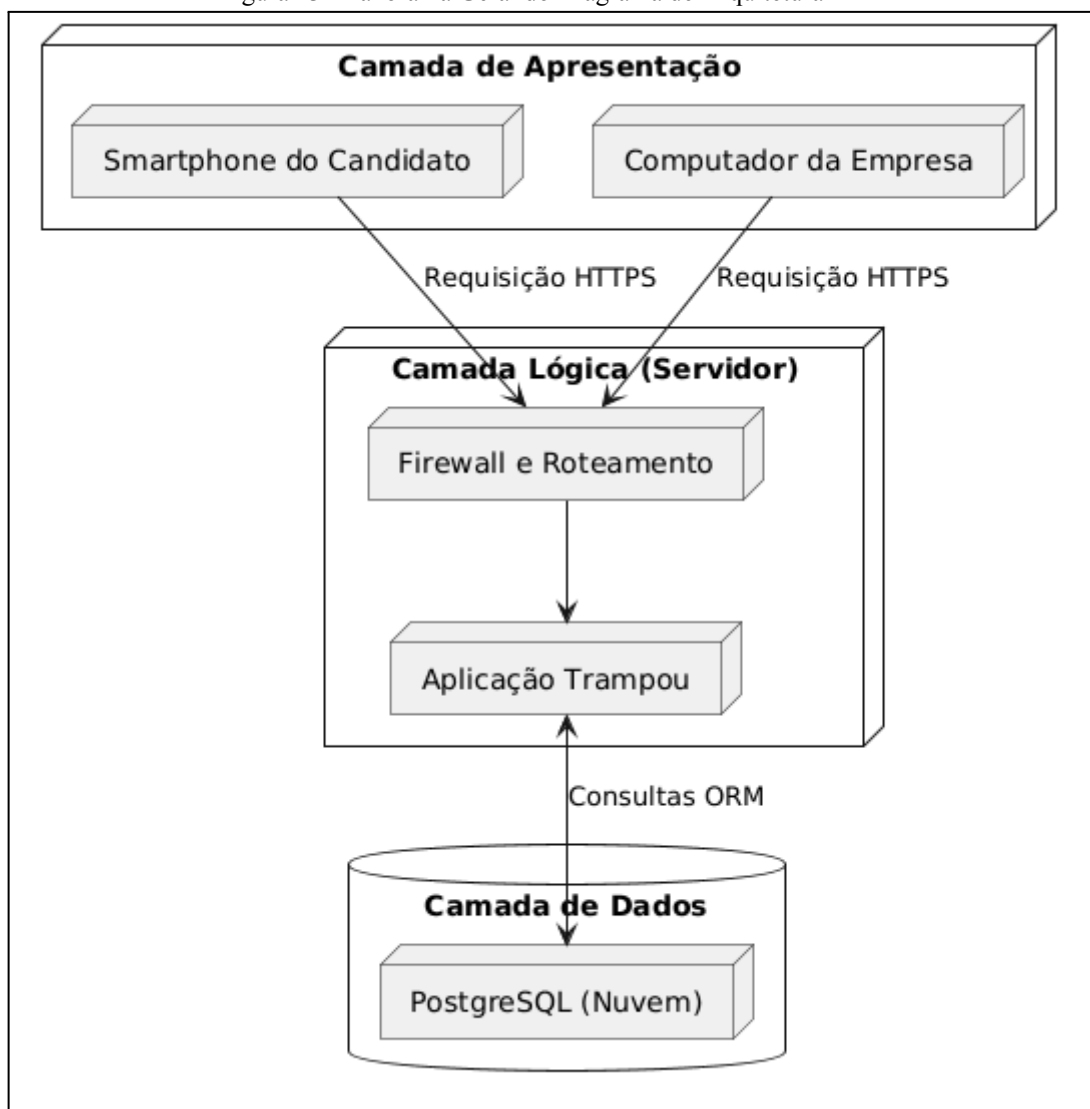
Figura 12 - Dimensão Direito do Panorama Geral do Diagrama de Componentes



Fonte: Produzido pelo autor

3.3.4 Diagrama de Arquitetura

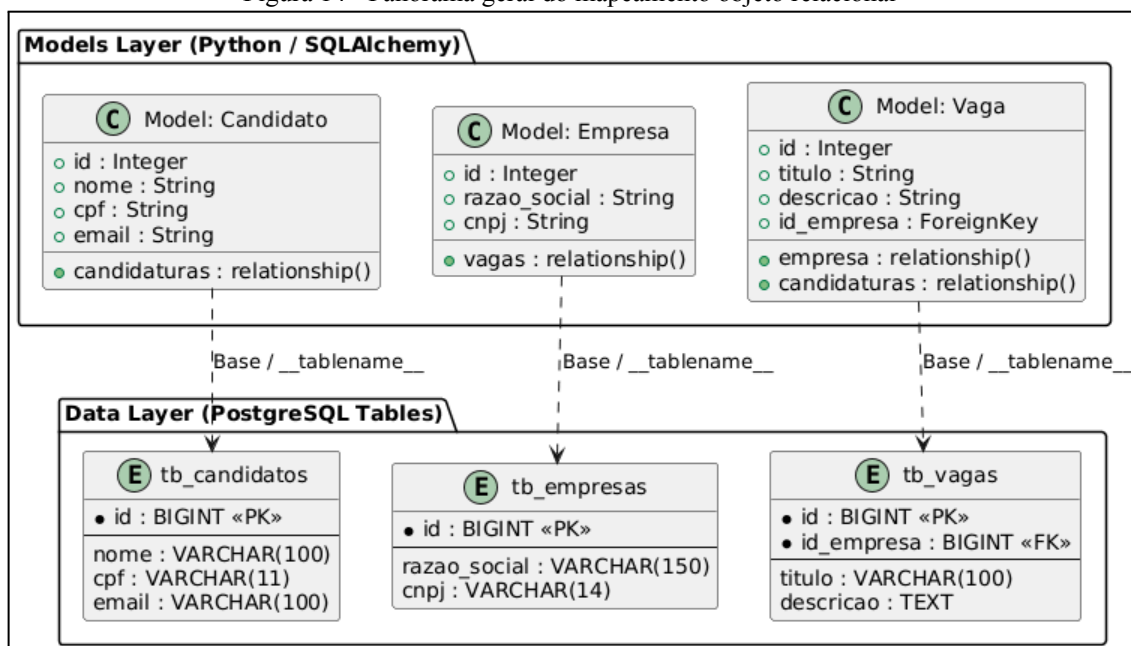
Figura 13 - Panorama Geral do Diagrama de Arquitetura



Fonte: Produzido pelo autor

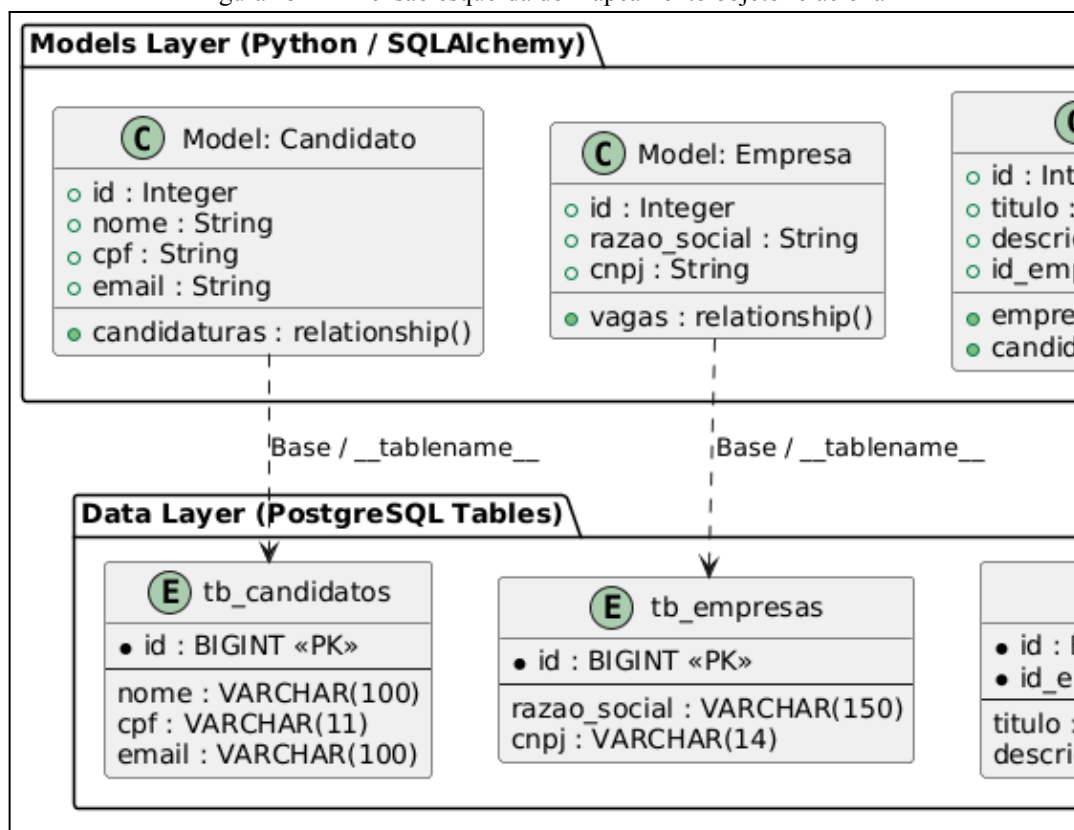
3.4 MAPEAMENTO OBJETO-RELACIONAL (ORM)

Figura 14 - Panorama geral do mapeamento objeto relacional



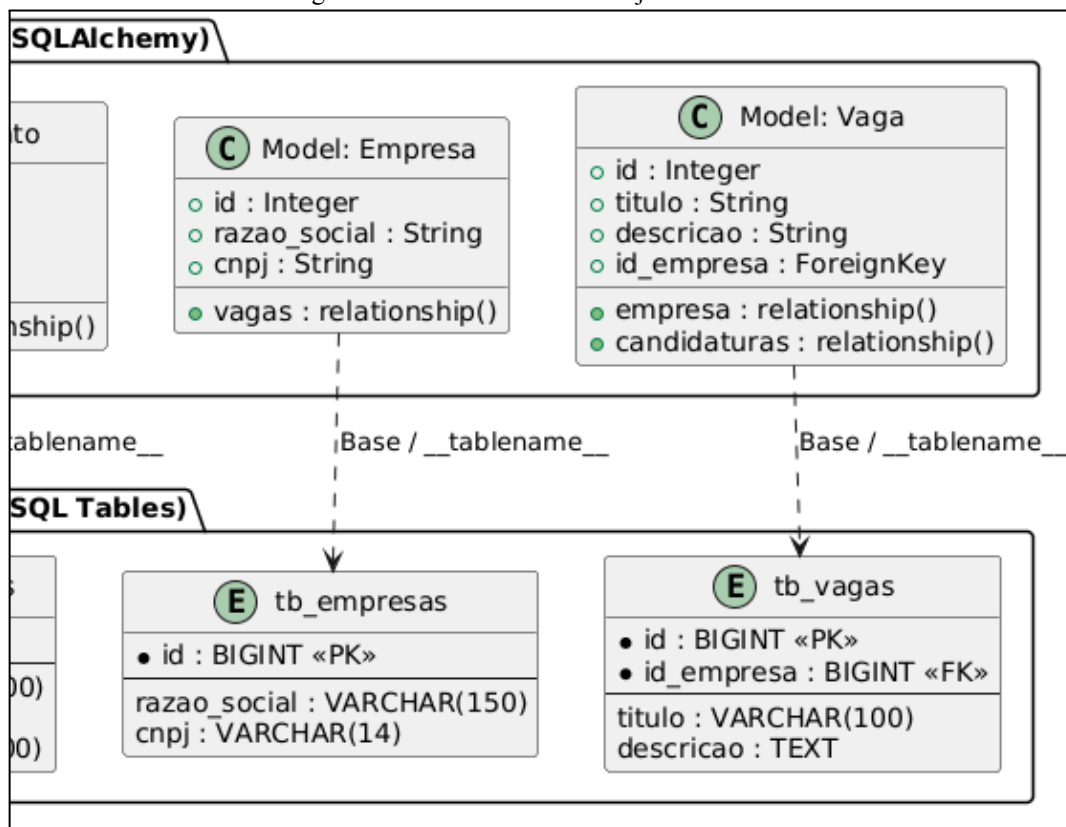
Fonte: Produzido pelo autor

Figura 15 - Dimensão esquerda do mapeamento objeto relacional



Fonte: Produzido pelo autor

Figura 16 - Panorama direito Objeto-Relacional



Fonte: Produzido pelo autor

DEI

4 DOCUMENTO DE ESPECIFICAÇÃO DE INTERFACES (DEI)

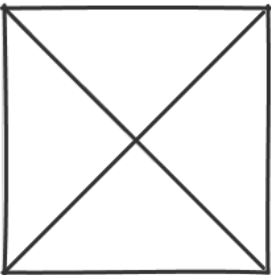
4.1 WIREFRAMES

Figura 1 - Tela 1: Mural de Vagas (Home)



Fonte: Produzido pelo autor

Figura 2 Tela 2: Autenticação (Login e Cadastro)



Login**Cadastro**

E-mail

Senha

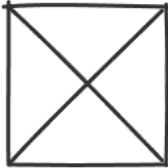
[Esqueci a senha](#)

Sou Candidato**Sou Empresa**

Entrar

Fonte: Produzido pelo autor

Figura 3 Tela 3: Perfil Digital do Candidato



Meu Perfil Digital
João da Silva

Dados Pessoais

Experiência

Educação

Em Análise (2)	Entrevista (1)	Recusada (0)
--------------------------	--------------------------	------------------------

Fonte: Produzido pelo autor

Figura 4 Tela 4: Upload de Currículo (Anexar PDF)

Upload de Currículo



Clique para selecionar PDF

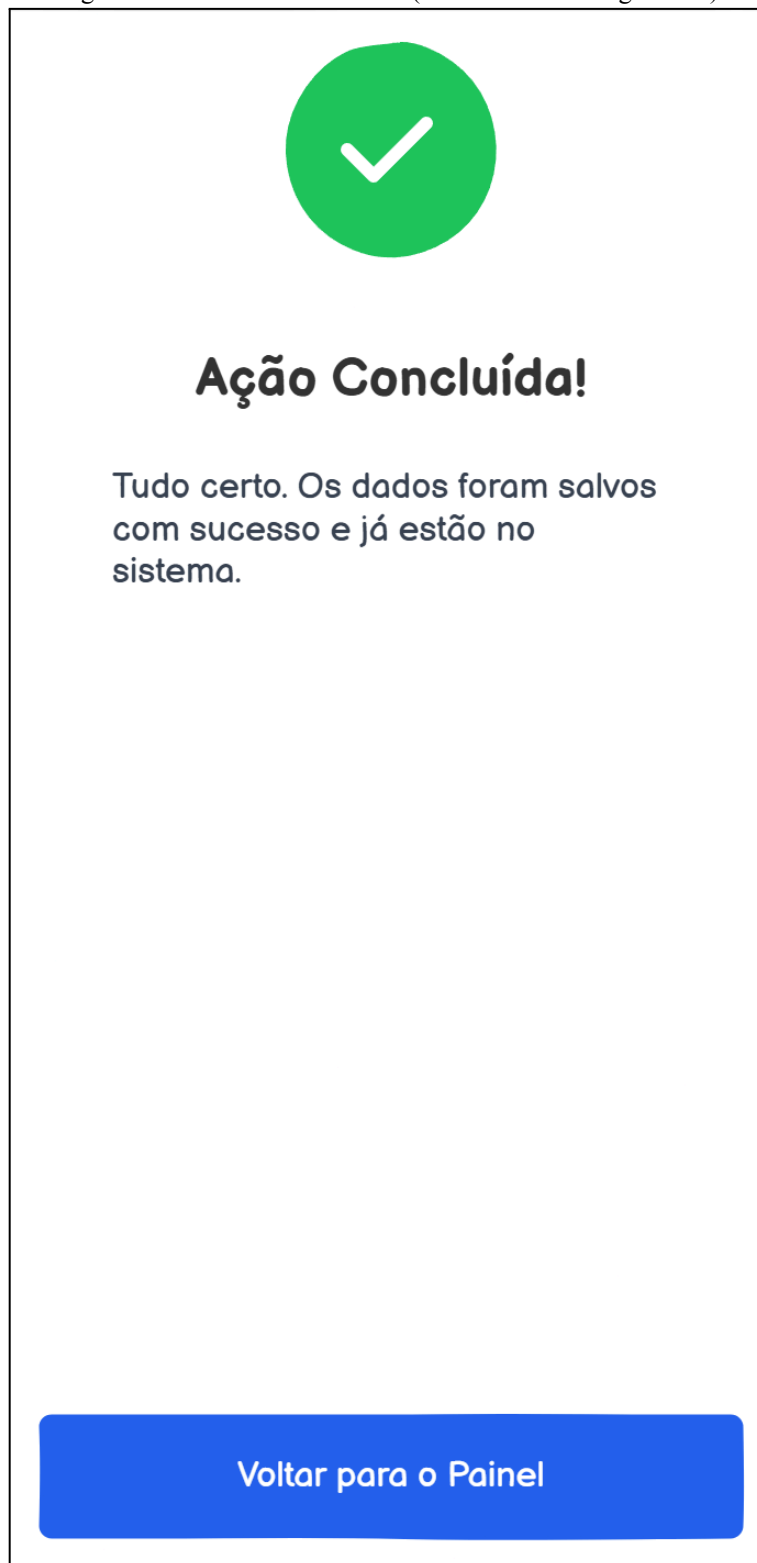
 curriculo_joao.pdf

Enviando...

Cancelar

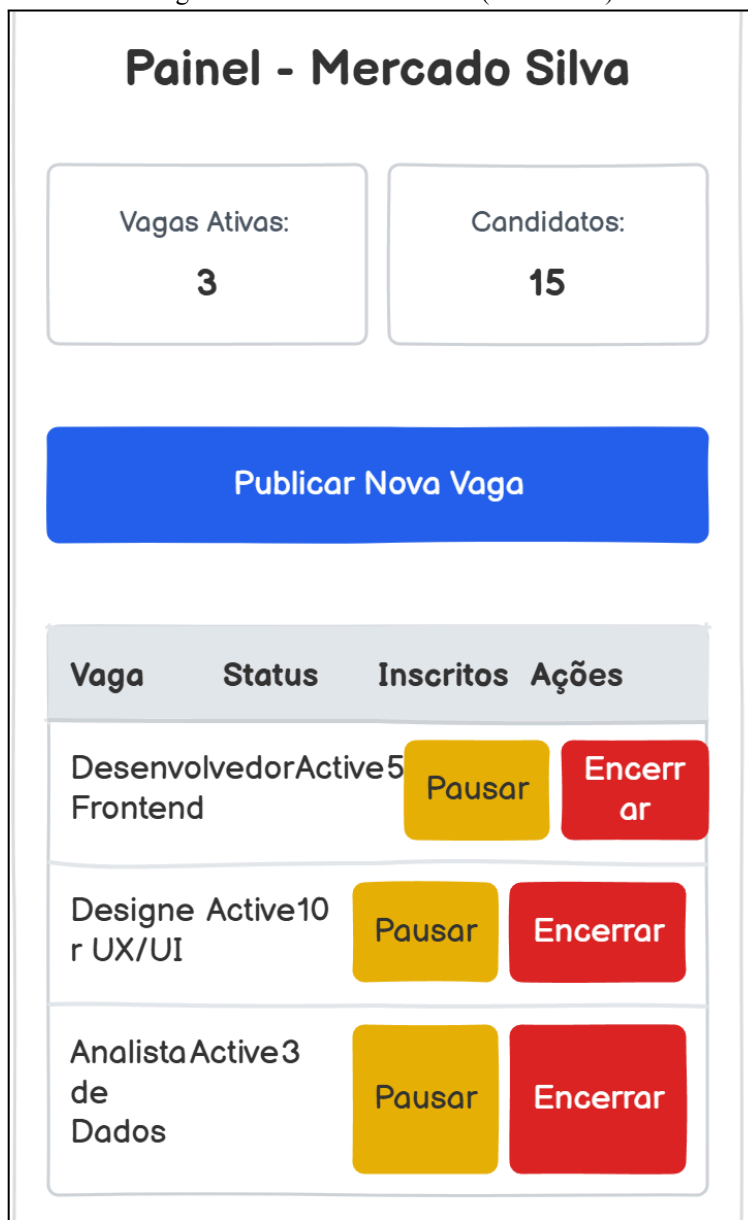
Fonte: Produzido pelo autor

Figura 5 Tela 5: Tela de Sucesso (Candidatura ou Vaga Salva)



Fonte: Produzido pelo autor

Figura 6 Tela 6: Painel Gestor (Dashboard)



Fonte: Produzido pelo autor

Figura 7 Tela 7: Criação/Edição de Vaga (Formulário)

Publicar Vaga

Título da Vaga

Salário (Opcional)

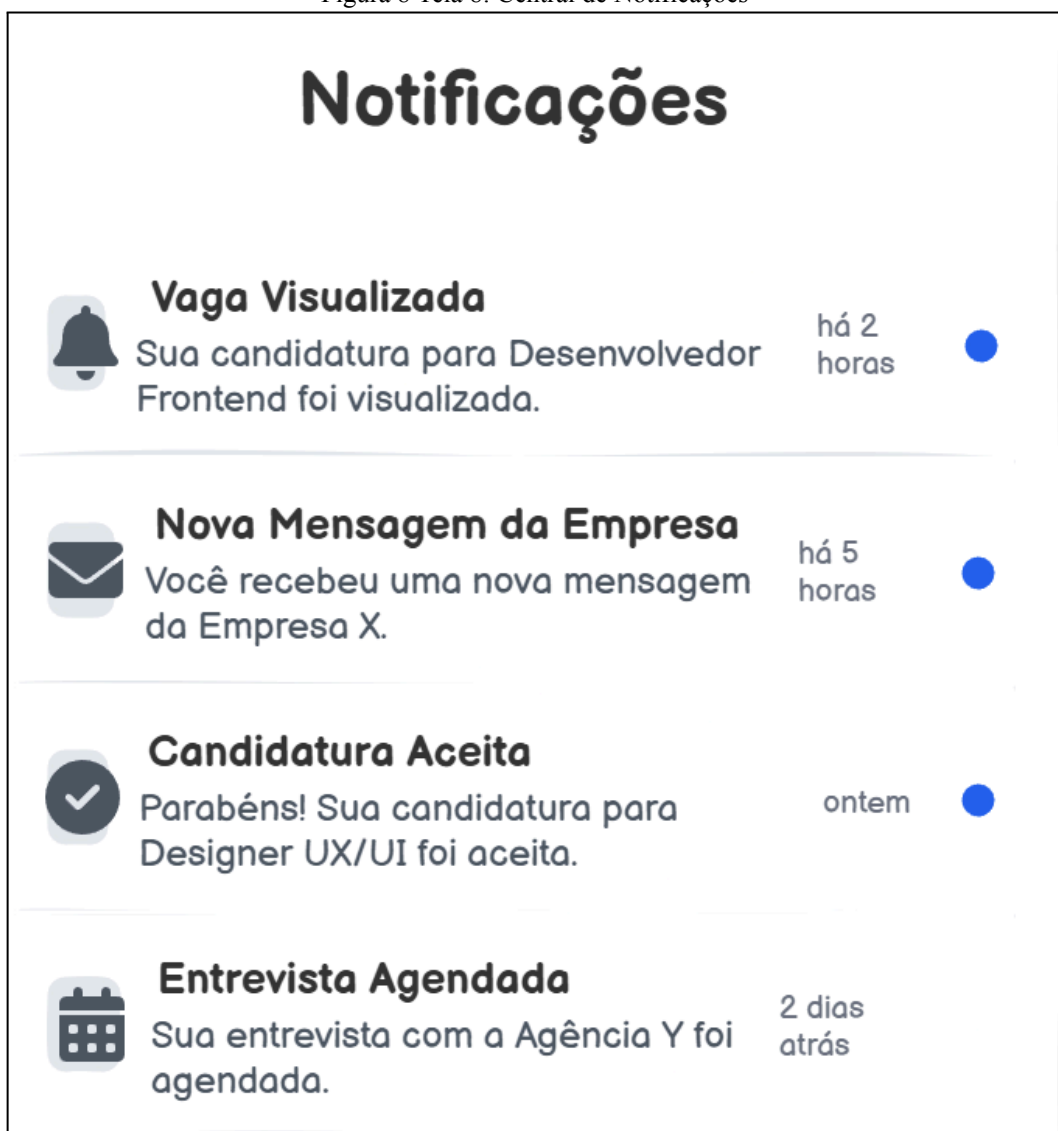
Requisitos

Cidade

CancelarPublicar Vaga

Fonte: Produzido pelo autor

Figura 8 Tela 8: Central de Notificações



Fonte: Produzido pelo autor

4.2 MOCKUPS

Figura 9 Tela 1: Mural de Vagas (Home)



Fonte: Produzido pelo autor

Figura 10 Tela 2: Autenticação (Login e Cadastro)

The image is a mobile app mockup for a screen titled 'Trampou'. The background is a light pink color. At the top center, the word 'Trampou' is written in a large, bold, dark brown font. Below the title, there are two tabs: 'Login' and 'Cadastro'. The 'Login' tab is selected, indicated by a dark brown underline. Below the tabs, there are two buttons: 'Sou Candidato' (dark brown with white text) and 'Sou Empresa' (light pink with dark brown text). Below these buttons are two input fields: 'E-mail' and 'Senha'. The 'Senha' field has a dark brown eye icon to its right. Below the 'Senha' field, there is a link that says 'Esqueci a senha'. At the bottom of the screen, there is a large dark brown button with the text 'Entrar' in white.

Trampou

Login Cadastro

Sou Candidato Sou Empresa

E-mail

Senha 

[Esqueci a senha](#)

Entrar

Fonte: Produzido pelo autor

Figura 11 Tela 3: Perfil Digital do Candidato



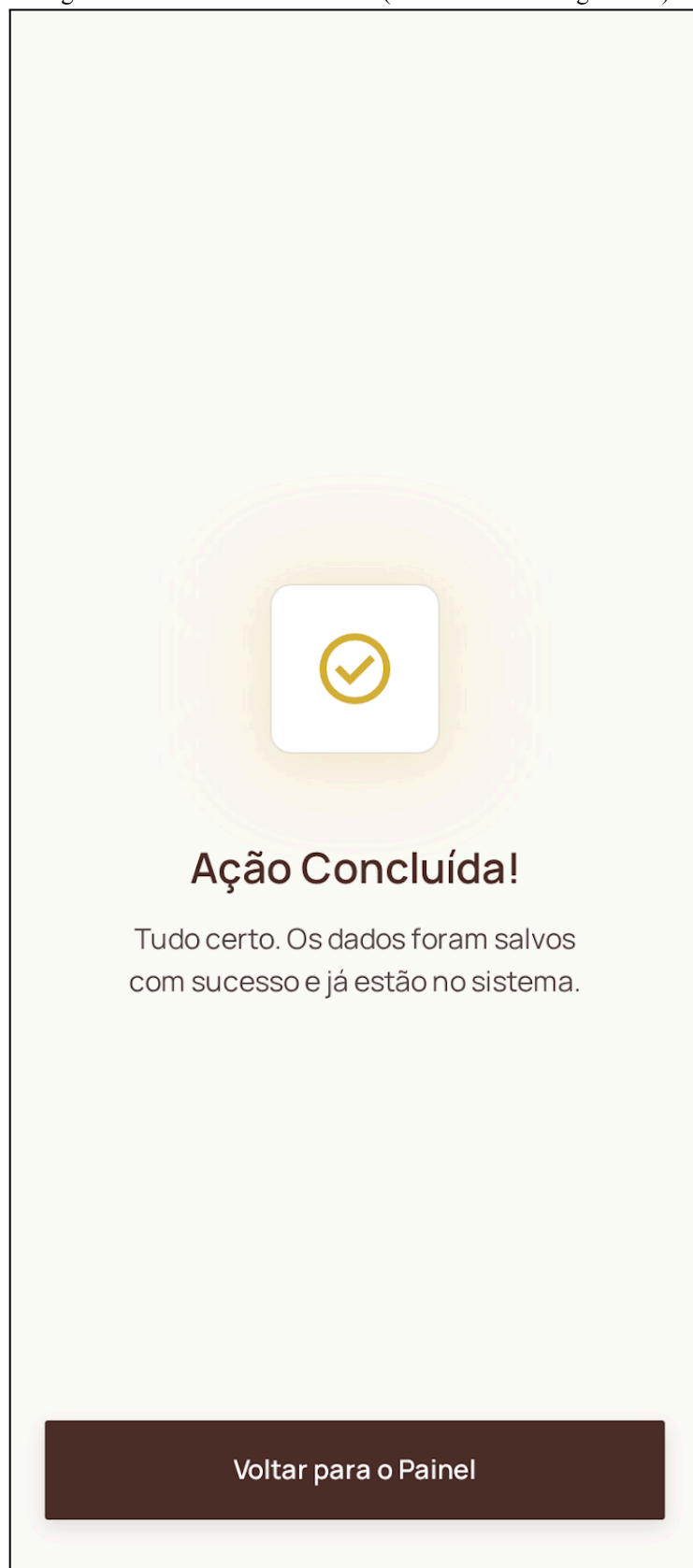
Fonte: Produzido pelo autor

Figura 12 Tela 4: Upload de Currículo (Anexar PDF)



Fonte: Produzido pelo autor

Figura 13 Tela 5: Tela de Sucesso (Candidatura ou Vaga Salva)



Fonte: Produzido pelo autor

Figura 14 Tela 6: Painel Gestor (Dashboard)



Fonte: Produzido pelo autor

Figura 15 Tela 7: Criação/Edição de Vaga (Formulário)



O formulário, intitulado "Publicar Vaga", possui um cabeçalho com uma seta de retorno e o título. O corpo do formulário contém quatro campos de entrada: "Título da Vaga", "Salário (Opcional)", "Requisitos" (uma área de texto maior) e "Cidade". Na base, há dois botões: "Cancelar" e "Publicar Vaga".

← **Publicar Vaga**

Fonte: Produzido pelo autor

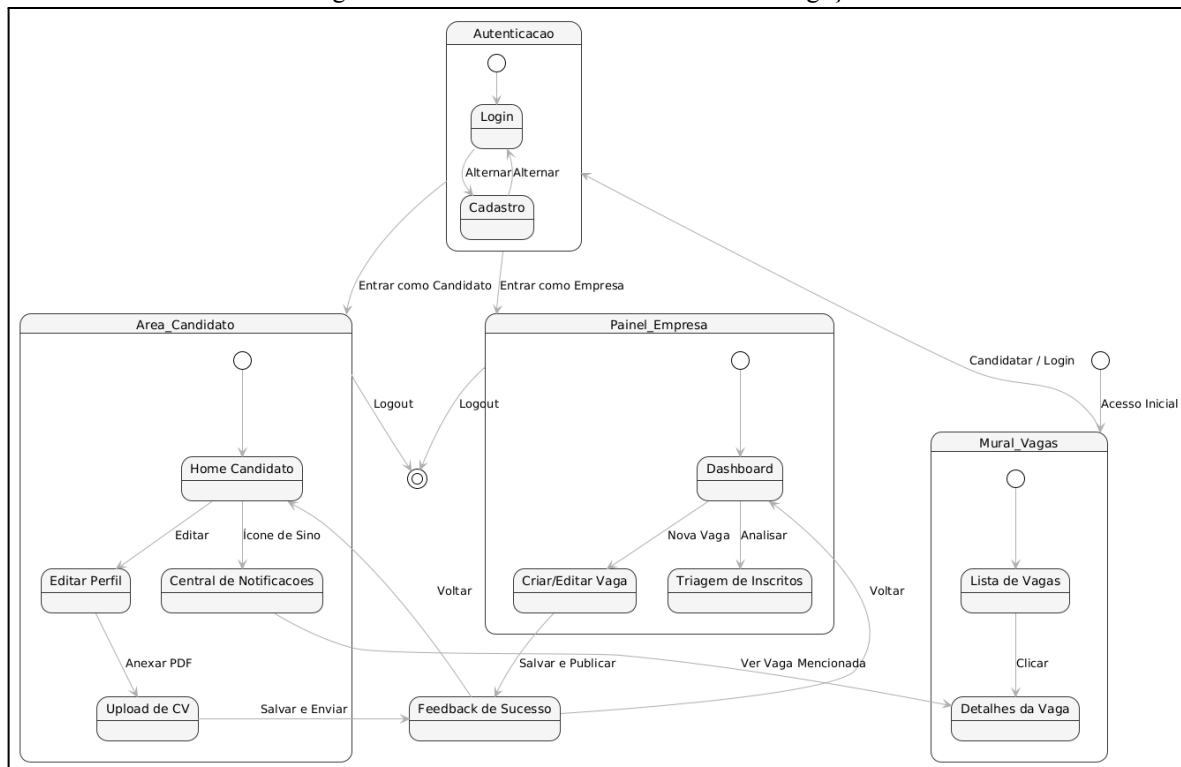
Figura 16 Tela 8: Central de Notificações



Fonte: Produzido pelo autor

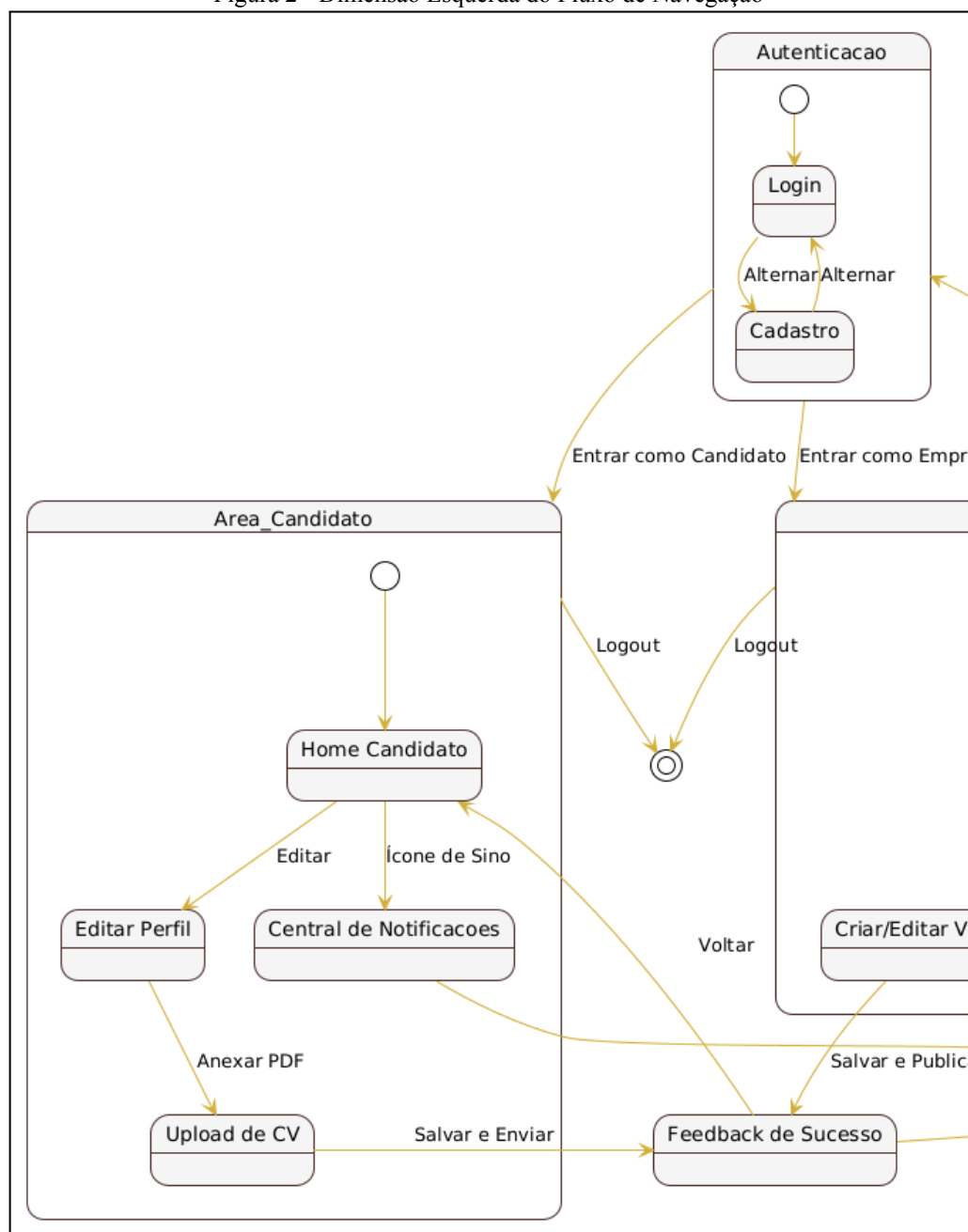
4.3 FLUXO DE NAVEGAÇÃO

Figura 17 - Dimensão Geral do Fluxo de Navegação



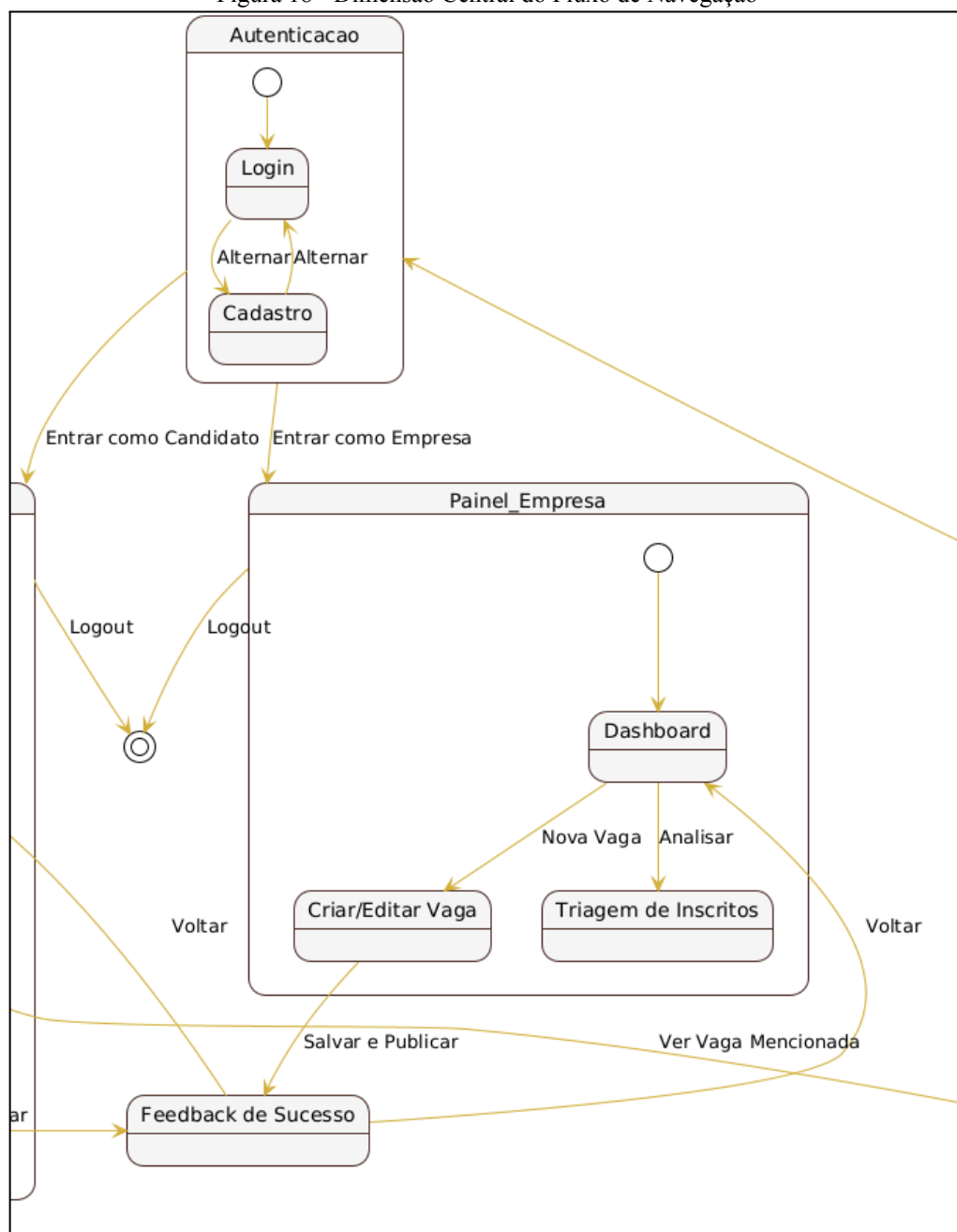
Fonte: Produzido pelo autor

Figura 2 - Dimensão Esquerda do Fluxo de Navegação



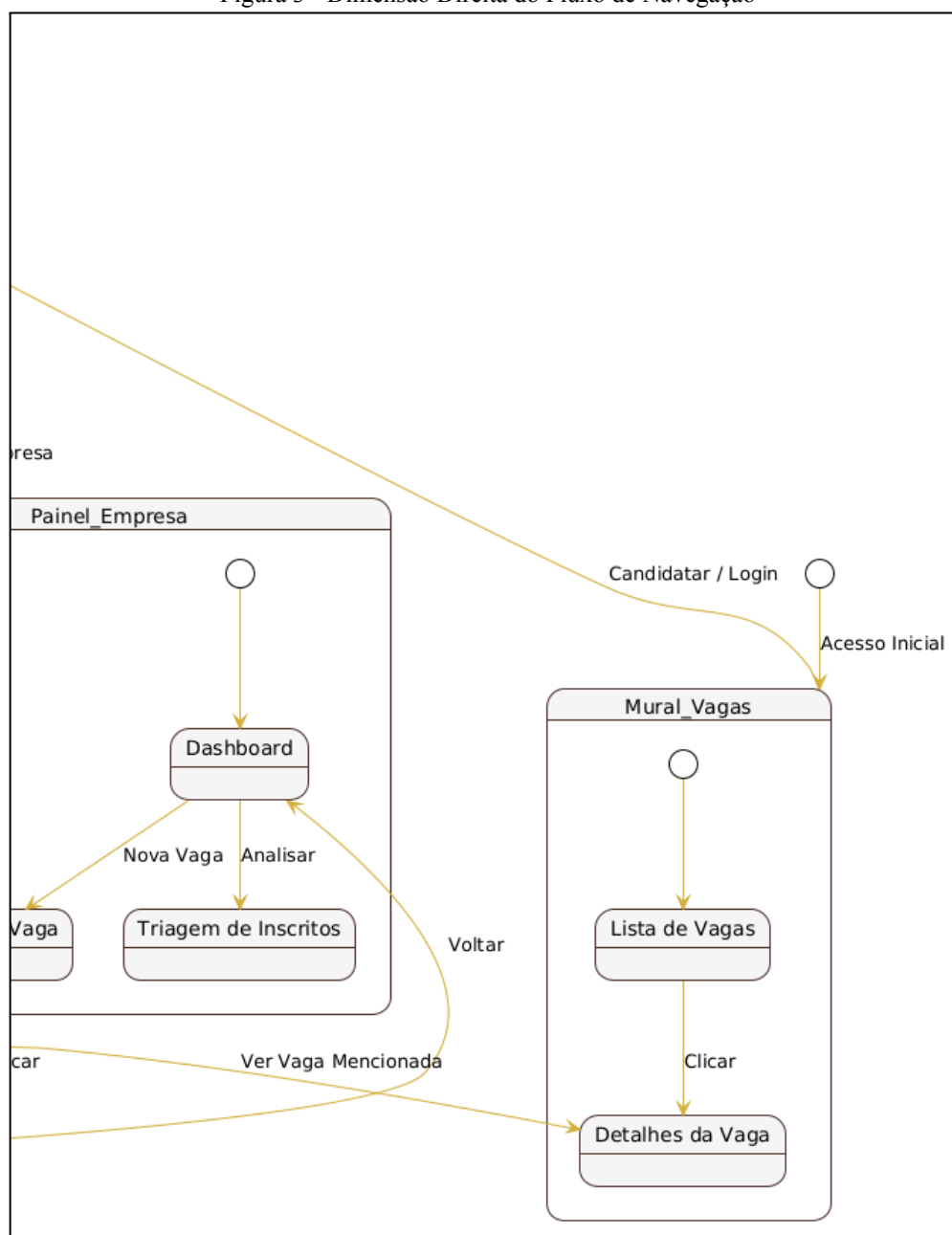
Fonte: Produzido pelo autor

Figura 18 - Dimensão Central do Fluxo de Navegação



Fonte: Produzido pelo autor

Figura 3 - Dimensão Direita do Fluxo de Navegação



Fonte: Produzido pelo autor

DT

5 DOCUMENTAÇÃO TÉCNICA

Este capítulo destina-se a documentar as decisões arquiteturais, as tecnologias adotadas e as convenções de código da plataforma Trampou, fornecendo uma visão clara e reprodutível do ambiente de desenvolvimento e produção.

5.1 ARQUITETURA DO SISTEMA

A plataforma foi desenvolvida sob o modelo de Arquitetura em Camadas (N-Tier Architecture), especificamente dividida em três camadas principais: Apresentação (Frontend), Aplicação (Backend) e Dados (Banco de Dados). Este modelo foi escolhido para garantir a separação de responsabilidades (Separation of Concerns), facilitando a manutenção, a escalabilidade e a testabilidade independente de cada módulo. A macroestrutura desta arquitetura encontra-se ilustrada no Diagrama de Arquitetura (Figura X - Seção 3.3.1).

5.1.1 Segmentação da Arquitetura

O código-fonte do *backend* foi organizado logicamente para refletir as divisões de responsabilidade:

- **Camada de Roteamento (Routers/Endpoints):** Responsável por interceptar as requisições HTTP (REST), definir os caminhos da API e direcionar as chamadas para os serviços apropriados.
- **Camada de Serviço (Services/Business Logic):** Contém a lógica central da aplicação (ex: regras para triagem de candidatos e validação de senhas). Ela orquestra as operações, sem conhecer os detalhes da infraestrutura do banco de dados.
- **Camada de Persistência (Models/Repositories):** Gerencia a comunicação direta com o banco de dados PostgreSQL através do mapeamento objeto-relacional (ORM), isolando a complexidade das consultas SQL do restante da aplicação.

5.2 TECNOLOGIAS UTILIZADAS

5.2.1 Frontend

A interface de usuário foi construída utilizando React.js (v18) integrado ao Next.js (v14) para garantir renderização otimizada e navegação fluida. A estilização visual foi inteiramente desenvolvida com Tailwind CSS (v3), um framework utilitário que permitiu a criação de componentes responsivos e alinhados aos protótipos de alta fidelidade sem a necessidade de extensos arquivos CSS externos.

5.2.2 Backend

O servidor de aplicação foi desenvolvido na linguagem Python (versão 3.12+), utilizando o *microframework* assíncrono FastAPI. O FastAPI foi adotado por sua altíssima performance estrutural, suporte nativo a operações assíncronas (ASGI) e pela capacidade de autogerar documentações interativas da API baseadas nos padrões OpenAPI.

5.2.3 Banco de Dados

A persistência de dados ocorre em um SGBD PostgreSQL (versão 15), escolhido por sua integridade transacional e robustez. A comunicação entre a aplicação Python e o banco é intermediada pela biblioteca SQLAlchemy, que atua como ORM para traduzir entidades em tabelas, garantindo segurança contra ataques de injeção de SQL.

5.2.4 Ferramentas de Apoio

O ciclo de desenvolvimento contou com as seguintes ferramentas:

- Git e GitHub: Controle de versionamento de código e repositório remoto.
- Insomnia / Postman: Teste manual e consumo das rotas da API REST.
- Unicorn: Servidor web ASGI de alta performance para executar a aplicação Python.
- Balsamiq Wireframes e Figma: Prototipação de baixa e alta fidelidade.
- Google Gemini: Assistência baseada em Inteligência Artificial para revisão de algoritmos, formatação de dados e refatoração de código.

5.2.5 Padrões Adotados

O projeto implementa padrões de projeto (Design Patterns) consolidados na Engenharia de Software:

- **Repository Pattern:** Abstrai as operações de banco de dados do SQLAlchemy, permitindo que a camada de serviço opere os dados sem acoplamento direto com as tabelas.
- **Dependency Injection (Injeção de Dependência):** Implementada de forma nativa através do sistema de `Depends` do FastAPI, provendo sessões de banco de dados apenas no momento de execução das rotas.
- **Singleton:** Aplicado na configuração de conexão com o banco de dados, garantindo que o pool de conexões seja instanciado apenas uma vez.

5.2.6 Boas Práticas e Convenções

A qualidade e a manutenibilidade do código foram asseguradas através da adoção de práticas rigorosas:

- **Clean Code e PEP 8:** O código-fonte segue estritamente a convenção de estilo *PEP 8* do Python. Variáveis e funções possuem nomes autoexplicativos em *snake_case* (ex: `buscar_candidato_por_cpf()`).
- **DTOs via Pydantic:** A transferência de dados entre o cliente e a API é validada através de *schemas* da biblioteca Pydantic. Isso garante a tipagem estrita e impede a exposição indevida de colunas sensíveis (como hashes de senhas) no retorno JSON.
- **Tratamento de Erros e Exceções:** Implementação de manipuladores globais de exceção (`Exception Handlers`) no FastAPI, retornando respostas HTTP padronizadas com mensagens amigáveis ao usuário final e omitindo *stack traces* do servidor.
- **Versionamento Semântico:** As entregas utilizam o padrão SemVer, com marcações de *tags* (ex: `v1.0.0`, `v1.1.0`) no Git.
- **Segurança Básica:** Configurações sensíveis (credenciais de banco e chaves secretas) são lidas através de variáveis de ambiente (`.env`). As senhas dos usuários são obrigatoriamente hashadas com algoritmos fortes (como BCrypt via Passlib) antes da persistência.

5.2.7 Requisitos de Infraestrutura

- Ambiente de Desenvolvimento: Sistema Operacional Windows, Linux ou macOS; mínimo de 8 GB de RAM; Python 3.12 ou superior instalado; Node.js (v20+) para o frontend.
- **Ambiente de Produção (Servidor):** Contêiner Linux executando o servidor Uvicorn, com alocação mínima de 1 GB de RAM para a API Python e 512 MB de RAM para o banco PostgreSQL, viável em plataformas de nuvem modernas (PaaS).

5.2.8 APIs e Integrações

- **Serviço de E-mail (SMTP):** Utilização de bibliotecas nativas ou integrações como `fastapi-mail` para o disparo assíncrono de e-mails transacionais, como confirmação de criação de conta e alertas para empresas.

5.2.9 Caracterização da API

A comunicação entre o frontend e o backend é realizada através de uma API arquitetada no padrão RESTful. As trocas de mensagens ocorrem estritamente via protocolo HTTP/HTTPS, utilizando o formato JSON (JavaScript Object Notation) para a serialização do corpo das requisições e respostas.

MDU

6 MANUAL DO USUÁRIO

Este capítulo apresenta as diretrizes de navegação e utilização da plataforma Trampou, com o objetivo de orientar o usuário final (candidatos e empresas) sobre o funcionamento das principais ferramentas do sistema. A interface foi projetada com foco em usabilidade e responsividade, garantindo uma curva de aprendizado rápida.

6.1 ACESSO INICIAL E AUTENTICAÇÃO

Ao acessar a plataforma, o usuário é direcionado à página inicial (Mural de Vagas), onde pode visualizar as oportunidades em destaque antes mesmo de realizar o login. Para interagir com o sistema, é necessário acessar a tela de Autenticação.

1. **Navegação para Login/Cadastro:** Clique no ícone de perfil localizado na barra de navegação inferior ou superior.
2. **Seleção de Perfil:** Na tela de autenticação, o usuário deve utilizar as abas superiores para escolher entre **Login** (se já possuir conta) ou **Cadastro** (para novos usuários).
3. **Identificação do Tipo de Conta:** Utilize os botões seletores para definir se o acesso será como **Sou Candidato** ou **Sou Empresa**. Preencha as credenciais (E-mail e Senha) e clique em "Entrar".

Figura 10 Tela 2: Autenticação (Login e Cadastro)

Trampou

Login Cadastro

Sou Candidato Sou Empresa

E-mail

Senha 

[Esqueci a senha](#)

Entrar

Fonte: Produzido pelo autor

6.2 VISÃO DO CANDIDATO

O módulo do candidato foi desenvolvido para facilitar a exposição do histórico profissional e agilizar o processo de candidatura com apenas um clique.

6.2.1 Configuração do Perfil Digital e Currículo

Antes de se candidatar a qualquer vaga, o usuário deve preencher suas informações.

1. Acesse a aba **Perfil**.
2. Navegue pelas seções estruturadas: **Dados Pessoais** (contato e endereço), **Experiência** (histórico profissional) e **Educação** (formação acadêmica).
3. **Upload de Currículo em PDF:** Caso deseje anexar um documento formatado, clique no botão de Upload. Clique na área de transferência para buscar o arquivo no seu dispositivo (tamanho máximo de 5MB) e aguarde a barra de progresso atingir 100%. Clique em "Cancelar" caso tenha selecionado o arquivo errado, ou confirme a operação.

Figura 12 Tela 4: Upload de Currículo (Anexar PDF)



Fonte: Produzido pelo autor

6.2.2 Busca e Candidatura

1. Acesse a aba **Início** para visualizar o Mural de Vagas.
2. Utilize a barra de pesquisa para filtrar oportunidades por cargo, empresa ou palavra-chave.
3. Ao encontrar uma vaga compatível, clique no botão **Candidatar-se**.
4. Se o perfil digital estiver completo, o sistema exibirá uma tela de confirmação ("Ação Concluída!"). Caso falte alguma informação obrigatória, o sistema emitirá um alerta para a finalização do cadastro.

Figura 13 Tela 5: Tela de Sucesso (Candidatura ou Vaga Salva)



Fonte: Produzido pelo autor

6.2.3 Acompanhamento e Notificações

O candidato não precisa entrar em contato direto com a empresa para saber o resultado da triagem.

1. **Painel de Status:** Na tela "Meu Perfil", o candidato visualiza um resumo quantitativo de suas candidaturas divididas por status (Em Análise, Entrevista, Recusada).
2. **Aba Notificações (Sino):** Todas as interações da empresa com o currículo geram um alerta nesta seção, informando desde a visualização do perfil até convites para entrevistas.

Figura 16 Tela 8: Central de Notificações



Fonte: Produzido pelo autor

6.3 VISÃO DA EMPRESA (GESTOR)

O módulo empresarial centraliza o recebimento de currículos, eliminando papéis e planilhas paralelas, permitindo que o gestor ou setor de RH controle todo o fluxo do processo seletivo em uma única tela.

6.3.1 Painel de Controle (Dashboard)

Ao realizar o login como "Empresa", o usuário é direcionado ao seu painel gerencial privativo.

1. **Métricas Rápidas:** Os painéis superiores exibem o total de "Vagas Ativas" e o volume geral de "Candidatos" inscritos.
2. **Gestão de Vagas:** A lista central exibe todas as oportunidades já criadas. Ao lado de cada vaga, existem botões de ação rápida para **Pausar** (ocultar a vaga temporariamente do mural) ou **Encerrar** (finalizar o processo seletivo definitivamente).

Figura 14 Tela 6: Painel Gestor (Dashboard)



Fonte: Produzido pelo autor

6.3.2 Publicação de Novas Vagas

Para anunciar uma oportunidade no comércio ou serviço local:

1. No Dashboard, clique no botão de destaque + **Publicar Nova Vaga**.
2. Preencha o formulário detalhado, informando obrigatoriamente o **Título da Vaga**, **Requisitos** e **Cidade**. O campo de **Salário** é opcional.
3. Clique em **Publicar Vaga**. A oportunidade ficará imediatamente visível no Mural Público de Vagas do sistema.

Figura 14 Tela 6: Painel Gestor (Dashboard)



Fonte: Produzido pelo autor

6.3.3 Triagem de Candidatos

Para analisar os interessados em uma vaga específica:

1. No Dashboard, clique sobre o título da vaga desejada.
2. O sistema listará todos os candidatos inscritos. Ao clicar em um nome, a empresa terá acesso integral ao Perfil Digital do candidato e ao seu PDF (caso anexado).
3. Utilize a ferramenta de atualização de status para mover o candidato no funil de seleção (ex: alterar de "Em Análise" para "Entrevista"). Esta ação dispara automaticamente uma notificação para o usuário.

Referências

REFERÊNCIAS

OBJECT MANAGEMENT GROUP (OMG). **Unified Modeling Language (UML) Specification**. Versão 2.5.1. 2017. Disponível em: <https://www.omg.org/spec/UML/2.5.1/PDF>. Acesso em: 18 maio 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation**. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 18 maio 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, I. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

TAILWIND LABS. **Tailwind CSS Documentation**. Disponível em: <https://tailwindcss.com/docs>. Acesso em: 18 maio 2026.

TURBAN, E.; POLLARD, C.; WOOD, G. **Tecnologia da Informação para Gestão: Estratégia, Apoio e Transformação no Mundo Digital**. 11. ed. Rio de Janeiro: LTC, 2021.

FASTAPI. **FastAPI Documentation**. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 18 maio 2026.

PYTHON SOFTWARE FOUNDATION. **Python 3.12 Documentation**. Disponível em: <https://docs.python.org/3/>. Acesso em: 18 maio 2026.

SQLALCHEMY. **SQLAlchemy Documentation**. Disponível em: <https://docs.sqlalchemy.org/>. Acesso em: 18 maio 2026.

OBJECT MANAGEMENT GROUP (OMG). **Unified Modeling Language (UML) Specification**. Versão 2.5.1. 2017. Disponível em: <https://www.omg.org/spec/UML/2.5.1/PDF>. Acesso em: 18 maio 2026.

VERCEL. **Vercel — Deploy**. Disponível em: <https://vercel.com>. Acesso em: 12 junho. 2026.

RENDER. **Render — Cloud hosting for developers (onrender)**. Disponível em: <https://render.com>. Acesso em: 12 jun. 2026.

PYTHON SOFTWARE FOUNDATION. **Python 3.12 Documentation**. Disponível em: <https://docs.python.org/3/>. Acesso em: 18 maio. 2026.

APENDICE

APÊNDICE A - CÓDIGO-FONTE DO DIAGRAMA ENTIDADE-RELACIONAMENTO
PRODUZIDO NO PLANTUML.

```

@startuml
    skinparam linetype ortho

    entity "Empresa" as empresa {
        * id_empresa : INT <<PK>>
        --
        * cnpj : VARCHAR(14)
        * razao_social : VARCHAR(100)
        endereco : VARCHAR(255)
    }

    entity "Candidato" as candidato {
        * id_candidato : INT <<PK>>
        --
        * cpf : VARCHAR(11)
        * nome : VARCHAR(100)
        resumo_profissional : TEXT
        contato : VARCHAR(50)
    }

    entity "Vaga" as vaga {
        * id_vaga : INT <<PK>>
        --
        * fk_empresa : INT <<FK>>
        * titulo : VARCHAR(100)
        descricao : TEXT
        status : VARCHAR(20)
    }

    entity "Candidatura" as candidatura {
        * id_candidatura : INT <<PK>>
        --
        * fk_candidato : INT <<FK>>
        * fk_vaga : INT <<FK>>
        status_triagem : VARCHAR(30)
        data_inscricao : DATETIME
    }

    empresa ||--o{ vaga : "publica"
    candidato ||--o{ candidatura : "realiza"
    vaga ||--o{ candidatura : "recebe"

@enduml

```

APÊNDICE B - CÓDIGO-FONTE DO DIAGRAMA DE MODELAGEM
COMPORTAMENTAL PRODUZIDO NO PLANTUML

```
@startuml
autonumber
actor "Candidato" as C
participant "Frontend (Interface Web)" as F
participant "Backend (API)" as B
database "Banco de Dados" as DB

C -> F: Clica no botão "Candidatar-se"
F -> B: Envia requisição (POST /candidatura)
B -> DB: Consulta dados do Candidato (SELECT)
DB --> B: Retorna dados do Perfil Digital

alt Perfil Incompleto
    B --> F: Retorna Erro (Campos obrigatórios vazios)
    F --> C: Exibe alerta: "Preencha seu perfil para continuar"
else Perfil Completo
    B -> DB: Registra a candidatura (INSERT)
    DB --> B: Confirmação de gravação
    B --> F: Retorna Sucesso (200 OK)
    F --> C: Exibe mensagem: "Candidatura realizada com sucesso!"
end
@enduml
```

APÊNDICE C - CÓDIGO-FONTE DO DIAGRAMA DE ESTADOS PRODUZIDO NO
PLANTUML

```
@startuml
[*] --> Nova : Candidato confirma interesse
Nova --> EmAnalise : Empresa visualiza perfil

state EmAnalise {
    [*] --> Analisando
}

EmAnalise --> Entrevista : Perfil selecionado
EmAnalise --> Recusada : Perfil não atende requisitos

Entrevista --> Aprovada : Candidato selecionado
Entrevista --> Recusada : Candidato não aprovado na entrevista

Aprovada --> [*] : Vaga preenchida
Recusada --> [*] : Processo encerrado
@enduml
```

APÊNDICE D - CÓDIGO-FONTE DO DIAGRAMA DE CASO DE USO PRODUZIDO NO PLANTUML

```

@startuml
left to right direction
skinparam packageStyle rectangle
skinparam usecase {
    BackgroundColor #f5f5f5
    BorderColor #4B2E2A
    ArrowColor #D4AF37
}
skinparam actor {
    BorderColor #4B2E2A
    BackgroundColor #D4AF37
}

actor "Visitante\n(Não Logado)" as Visitante
actor "Candidato\n(Autenticado)" as Candidato
actor "Empresa\n(Autenticada)" as Empresa

rectangle "Plataforma Trampou" {
    usecase "Cadastrar Conta" as UC1
    usecase "Pesquisar Vagas" as UC2
    usecase "Gerenciar Perfil Digital" as UC3
    usecase "Realizar Candidatura" as UC4
    usecase "Acompanhar Status de Triagem" as UC5
    usecase "Gerenciar Perfil da Empresa" as UC6
    usecase "Publicar Nova Vaga" as UC7
    usecase "Gerenciar Vagas (Pausar/Encerrar)" as UC8
    usecase "Triar Candidatos Inscritos" as UC9
}

Visitante --> UC1
Visitante --> UC2

Candidato --> UC2
Candidato --> UC3
Candidato --> UC4
Candidato --> UC5

Empresa --> UC6
Empresa --> UC7
Empresa --> UC8
Empresa --> UC9
@enduml

```

APÊNDICE E - CÓDIGO-FONTE DO DIAGRAMA DE CLASSES PRODUZIDO NO PLANTUML

```

@startuml
abstract class Usuario {
    + id : INT
    + email : VARCHAR
    + senha : VARCHAR
    + login()
    + logout()
}

class Empresa {
    + cnpj : VARCHAR
    + razao_social : VARCHAR
    + gerenciarPerfil()
}

class Candidato {
    + cpf : VARCHAR
    + nome : VARCHAR
    + escolaridade : VARCHAR
    + editarPerfilDigital()
}

class Vaga {
    + id : INT
    + titulo : VARCHAR
    + descricao : TEXT
    + status : VARCHAR
    + pausarVaga()
    + encerrarVaga()
}

class Candidatura {
    + status_triagem : VARCHAR
    + data_inscricao : DATETIME
    + atualizarStatus()
}

Usuario <|-- Empresa
Usuario <|-- Candidato
Empresa "1" --> "*" Vaga : publica
Candidato "1" --> "*" Candidatura : realiza

```

Vaga "1" --> "*" Candidatura : recebe
@enduml

APÊNDICE F - CÓDIGO-FONTE DO DIAGRAMA DE COMPONENTES PRODUZIDO NO PLANTUML

```
@startuml
package "Frontend - Interface Web" {
    [Módulo do Mural]
    [Módulo do Empregador]
    [Módulo do Candidato]
}

package "Backend - API RESTful" {
    [Gerenciador de Rotas]
    [Regras de Negócio]
    [Autenticação e Segurança]
}

database "Database" {
    [Banco de Dados Relacional]
}

[Frontend - Interface Web] <--> [Backend - API RESTful] : Requisições HTTP / JSON
[Backend - API RESTful] <--> [Banco de Dados Relacional] : Leitura e Gravação (ORM)
@enduml
```

APÊNDICE G - CÓDIGO-FONTE DO DIAGRAMA DE ARQUITETURA
PRODUZIDO NO PLANTUML

```
@startuml
node "Camada de Apresentação" {
    node "Smartphone do Candidato" as mobile
    node "Computador da Empresa" as desktop
}

node "Camada Lógica (Servidor)" {
    node "Firewall e Roteamento" as fw
    node "Aplicação Trampou" as app
    fw --> app
}

database "Camada de Dados" {
    node "PostgreSQL (Nuvem)" as db
}

mobile --> fw : Requisição HTTPS
desktop --> fw : Requisição HTTPS
app <--> db : Consultas ORM
@enduml
```

APÊNDICE H - CÓDIGO-FONTE DO DIAGRAMA DE MAPEAMENTO ORM (RELACIONAL) PRODUZIDO NO PLANTUML

@startuml

title Mapeamento Objeto-Relacional (ORM) - Python/SQLAlchemy - Trampou

```
package "Models Layer (Python / SQLAlchemy)" {
    class "Model: Candidato" as Cand {
        + id : Integer
        + nome : String
        + cpf : String
        + email : String
        + candidaturas : relationship()
    }
}
```

```
class "Model: Empresa" as Emp {
    + id : Integer
    + razao_social : String
    + cnpj : String
    + vagas : relationship()
}
```

```
class "Model: Vaga" as Vaga {
    + id : Integer
    + titulo : String
    + descricao : String
    + id_empresa : ForeignKey
    + empresa : relationship()
    + candidaturas : relationship()
}
}
```

```
package "Data Layer (PostgreSQL Tables)" {
    entity "tb_candidatos" as tb_cand {
        * id : BIGINT <<PK>>
        --
        nome : VARCHAR(100)
        cpf : VARCHAR(11)
        email : VARCHAR(100)
    }
}
```

```
entity "tb_empresas" as tb_emp {
    * id : BIGINT <<PK>>
    --
    razao_social : VARCHAR(150)
```

```

    cnpj : VARCHAR(14)
}

```

```

entity "tb_vagas" as tb_vaga {
    * id : BIGINT <<PK>>
    * id_empresa : BIGINT <<FK>>
    --
    titulo : VARCHAR(100)
    descricao : TEXT
}
}

```

```

Cand ..> tb_cand : Base / __tablename__
Emp ..> tb_emp : Base / __tablename__
Vaga ..> tb_vaga : Base / __tablename__

```

```

@enduml

```